

Design and Analysis of a Four-Quadrant DC Motor Drive Using an H-Bridge Converter and Cascaded PI Control

1. Introduction

1.1 DC Motors and the Need for Speed Regulation

A motor is an electromechanical device that converts electrical energy into mechanical energy. In the case of a DC motor, the supplied voltage and current are DC in nature. DC motors are widely used in industrial and everyday applications such as elevators, cranes, conveyor belts, rolling mills, electric vehicles and numerous motion control systems where accurate speed regulation is required.

For this project, a separately excited DC motor is considered. Before designing any controller, it is important to first understand why the speed of a motor changes.

The mechanical dynamics of a motor are governed by

$$T_e = T_L + J \frac{d\omega_m}{dt}$$

or equivalently,

$$T_e - T_L = J \frac{d\omega_m}{dt}$$

where T_e is the electromagnetic torque developed by the motor, T_L is the load torque, J is the total moment of inertia and ω_m is the motor speed.

This equation provides an important insight. Whenever the electromagnetic torque and load torque are unequal, the motor speed must change. For example, if the load torque suddenly increases while the motor torque remains unchanged,

$$T_e - T_L < 0$$

which causes

$$\frac{d\omega_m}{dt} < 0$$

and therefore the motor begins to slow down.

To understand what happens next, consider the armature voltage equation of a DC motor:

$$V = i_a R_a + L_a \frac{di_a}{dt} + K_e \omega_m$$

At steady state,

$$\frac{di_a}{dt} = 0$$

and therefore

$$V = i_a R_a + K_e \omega_m$$

When the motor speed decreases, the back EMF also decreases. Since the supply voltage remains unchanged, the armature current automatically increases. Because motor torque is proportional to armature current, the motor develops additional torque and eventually balances the new load torque.

At first glance this may seem sufficient. The motor slowed down, the current increased, the torque increased and the system reached a new equilibrium. However, the new equilibrium generally occurs at a different speed.

This behaviour becomes problematic in practical applications. Consider an elevator carrying a single passenger. If ten additional passengers enter the elevator, the load torque changes significantly. Similarly, on a conveyor belt, heavier packages create additional load on the motor. If motor speed changes every time the load changes, system performance becomes inconsistent and productivity may suffer.

While maintaining the correct final speed is important, the time taken to reach that speed is equally important. In many applications, a delay of a few seconds may be acceptable. However, in applications such as textile mills, even a small temporary variation in speed or tension can cause threads to stretch excessively or break, leading to production losses. In such cases, it is not sufficient for the motor to eventually reach the desired speed; it must reach it quickly and with minimal deviation. Therefore, practical speed control systems must consider not only the final operating speed but also the dynamic response of the system.

Therefore, allowing the motor to naturally settle is not enough. What is required is a mechanism that continuously monitors the speed and automatically adjusts the motor input so that the desired speed is maintained despite disturbances and load variations.

1.2 Fundamentals of Feedback Control

The next question is straightforward: how do we know whether the motor is operating at the desired speed?

The simplest approach is to compare the desired speed with the actual speed. Let

$$\omega_{ref}$$

represent the desired speed and

$$\omega_m$$

represent the actual motor speed.

The difference between these two quantities is known as the error signal:

$$e = \omega_{ref} - \omega_m$$

The error signal immediately tells us two important things:

1. How far the system is from the desired operating point.
2. Whether the speed needs to increase or decrease.

If the actual speed is lower than the reference speed, the error becomes positive and corrective action must increase the motor speed. Conversely, if the actual speed is higher than the reference speed, the error becomes negative and corrective action must reduce the motor speed.

Knowing the error alone is not enough. We still need a mechanism that can automatically convert this error into a suitable control action. This is the role of a controller.

A controller can be viewed as a mathematical function that receives an error signal as input and produces a control signal as output:

$$u = f(e)$$

The controller continuously observes the error and adjusts the system input in an attempt to reduce that error to zero. This concept forms the basis of closed-loop or feedback control systems used throughout modern industry.

1.3 Proportional and Integral Control

One of the simplest controllers is the proportional (P) controller. In a proportional controller, the control action is directly proportional to the error signal:

$$u = K_p e$$

This relationship is intuitive. If the error is large, the controller produces a large corrective action. If the error is small, the corrective action is also small.

As the motor approaches the desired operating point, the controller naturally reduces its output. Consequently, proportional control significantly improves system response and is simple to implement.

However, proportional control has an important limitation. In many practical systems, a small error must remain present to generate the control effort required to balance disturbances and load torque. As a result, the system often settles with a non-zero steady-state error.

Increasing the proportional gain can reduce this error, but it cannot eliminate it completely. Furthermore, excessively large proportional gains may cause oscillations and poor stability.

To eliminate the remaining error, integral action is introduced. The integral controller continuously accumulates past error values and generates a control action proportional to the accumulated error:

$$u = K_i \int e(t) dt$$

Unlike proportional control, the integral controller possesses memory. Even a small persistent error continues accumulating over time, forcing the controller to increase its output until the error is removed.

Combining proportional and integral actions results in the PI controller:

$$G_c(s) = K_p + \frac{K_i}{s}$$

The proportional component provides rapid response while the integral component eliminates steady-state error. Because of this balance between simplicity and performance, PI controllers are among the most widely used controllers in industrial motor drive systems.

1.4 Choice of Separately Excited DC Motor

In this project, a separately excited DC motor is used. In this type of machine, the field winding is supplied from an independent source, resulting in an approximately constant field flux.

Under this condition, the motor torque and back EMF are given by

$$T_e = K_t I_a$$

and

$$E_b = K_e \omega_m$$

respectively.

These equations reveal an important advantage. The motor torque becomes directly proportional to armature current, while the back EMF becomes directly proportional to motor speed.

As a result, torque and speed can be analysed independently to a large extent, greatly simplifying controller design. This makes the separately excited DC motor an excellent platform for studying speed control systems and developing advanced control strategies.

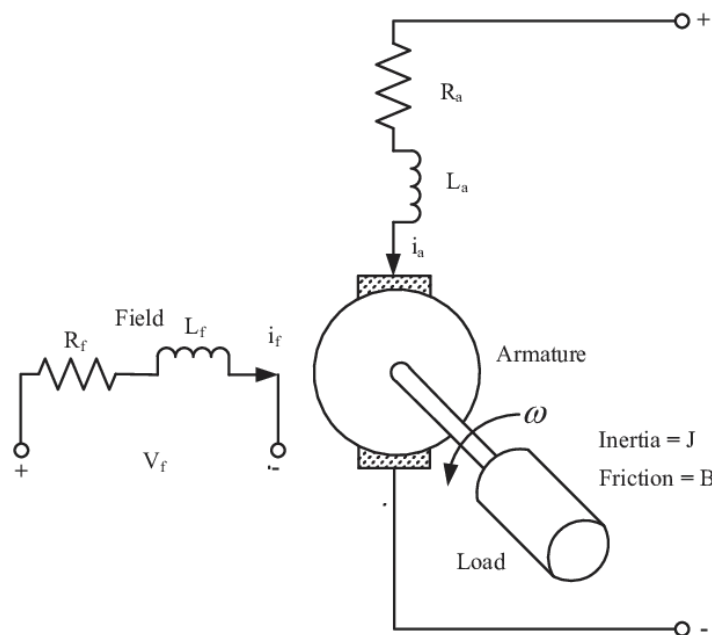


Figure 1.1: Separately Excited DC Motor Configuration

Source: ResearchGate [1]

1.5 Project Objective

The objective of this project is to design, analyse and implement a closed-loop speed control system for a separately excited DC motor using PWM-based voltage control.

The work begins with mathematical modelling of the motor and development of basic control concepts. Controller design is then carried out using frequency-domain techniques and loop shaping. The control system is progressively improved through the introduction of cascaded speed-current control, current limiting, anti-windup compensation, feedforward action and current filtering.

Finally, the work is extended from a one-quadrant drive to a four-quadrant H-bridge drive capable of regenerative braking, bidirectional torque production and bidirectional speed operation. The performance of the resulting system is evaluated through reference tracking, disturbance rejection and regenerative braking tests.

2. Mathematical Modelling

Before designing any controller, it is necessary to develop a mathematical model of the motor. A controller can only improve the behaviour of a system if the behaviour of the system itself is known. Therefore, the first step is to establish the relationship between the applied armature voltage and the resulting motor speed.

The dynamics of a separately excited DC motor can be described using two fundamental equations:

1. Mechanical Dynamics

2. Electrical Dynamics

Together, these equations describe how electrical energy supplied to the motor is converted into mechanical motion.

2.1 Mechanical Dynamics of the DC Motor

The rotational dynamics of a motor are governed by Newton's Second Law for rotational systems. The developed electromagnetic torque must balance the load torque and the torque required to accelerate the rotating inertia.

Thus,

$$T_e = T_L + J \frac{d\omega_m}{dt}$$

or equivalently,

$$T_e - T_L = J \frac{d\omega_m}{dt}$$

where T_e is the electromagnetic torque developed by the motor, T_L is the load torque, J is the combined motor-load inertia and ω_m is the motor speed.

This equation shows that whenever the load torque changes, the motor speed is forced to change unless the developed electromagnetic torque changes accordingly.

In practice, the total load torque acting on a motor consists of multiple components. These components are summarized in Table 2.1.

Table 2.1: Components of Load Torque

Torque Component	Expression	Description
Mechanical Load Torque	T_m	Torque required by the external mechanical load
Windage Torque	$T_w = C\omega_m^2$	Torque due to air drag and windage losses
Viscous Friction Torque	$T_v = B\omega_m$	Friction proportional to speed
Coulomb Friction Torque	T_c	Constant friction opposing motion
Stiction Torque	T_s	Static friction acting at very low or zero speed

Therefore, the total load torque may be represented as

$$T_L = T_m + B\omega_m + C\omega_m^2 + T_c + T_s$$

For controller design, higher-order friction effects are generally neglected and only the dominant viscous friction component is retained. The mechanical dynamics can therefore be approximated as

$$T_e = T_L + B\omega_m + J \frac{d\omega_m}{dt}$$

This simplified model captures the dominant behaviour of the motor while keeping the mathematical analysis manageable.

2.2 Electrical Dynamics of the DC Motor

The electrical behaviour of the armature circuit is obtained by applying Kirchhoff's Voltage Law around the armature loop.

The armature voltage equation is given by

$$V_a = R_a i_a + L_a \frac{di_a}{dt} + E_b$$

where V_a is the applied armature voltage, R_a is the armature resistance, L_a is the armature inductance, i_a is the armature current and E_b is the back electromotive force.

The first term represents the resistive voltage drop while the second term represents the inductive voltage drop. The final term, known as the Back EMF, is generated due to motor rotation.

According to Faraday's Law of Electromagnetic Induction, whenever a conductor moves through a magnetic field an induced voltage is generated. Since this induced voltage opposes the applied armature voltage, it is referred to as Back EMF.

For a separately excited DC motor operating with approximately constant field flux,

$$E_b = K_e \omega_m$$

where K_e is the back EMF constant.

Similarly, the electromagnetic torque produced by the motor is proportional to armature current and is given by

$$T_e = K_t i_a$$

where K_t is the motor torque constant.

These two equations provide an important physical insight:

- Armature Voltage controls Armature Current.
- Armature Current controls Electromagnetic Torque.
- Electromagnetic Torque controls Acceleration.
- Acceleration controls Motor Speed.

Thus, voltage indirectly controls motor speed through the electrical and mechanical dynamics of the machine.

Substituting

$$E_b = K_e \omega_m$$

into the armature voltage equation gives

$$V_a = R_a i_a + L_a \frac{di_a}{dt} + K_e \omega_m$$

which forms the complete electrical model of the motor.

2.3 Complete Motor Model

The electrical and mechanical equations derived in the previous sections can now be rearranged into forms that clearly reveal the dynamic behaviour of the motor.

$$T_e - T_L = B\omega_m + J \frac{d\omega_m}{dt}$$

Rearranging,

$$T_e - T_L = B\omega_m \left(1 + \frac{J}{B} \frac{d}{dt} \right)$$

Defining the mechanical time constant as

$$T_m = \frac{J}{B}$$

gives

$$T_e - T_L = B\omega_m \left(1 + T_m \frac{d}{dt} \right)$$

Taking the Laplace Transform,

$$T_e(s) - T_L(s) = B\omega_m(s)(1 + sT_m)$$

Therefore,

$$\omega_m(s) = (T_e(s) - T_L(s)) \frac{1/B}{1 + sT_m}$$

This equation represents the mechanical subsystem of the motor.

The mechanical time constant T_m determines how rapidly the motor speed responds to changes in electromagnetic torque.

Similarly, starting with the electrical equation,

$$V_a = R_a i_a + L_a \frac{di_a}{dt} + E_b$$

Rearranging,

$$V_a - E_b = R_a i_a + L_a \frac{di_a}{dt}$$

Factoring R_a ,

$$V_a - E_b = R_a i_a \left(1 + \frac{L_a}{R_a} \frac{d}{dt} \right)$$

Defining the electrical time constant as

$$T_a = \frac{L_a}{R_a}$$

gives

$$V_a - E_b = R_a i_a \left(1 + T_a \frac{d}{dt} \right)$$

Taking the Laplace Transform,

$$V_a(s) - E_b(s) = R_a I_a(s) (1 + sT_a)$$

Therefore,

$$I_a(s) = (V_a(s) - E_b(s)) \frac{1/R_a}{1 + sT_a}$$

This equation represents the electrical subsystem of the motor.

An important observation can immediately be made. For most DC motors,

$$T_m \gg T_a$$

This means that the electrical dynamics are significantly faster than the mechanical dynamics. In other words, armature current reaches its steady-state value much faster than motor speed.

This separation of time scales is extremely useful in controller design and later motivates the use of cascaded current and speed control loops.

Using

$$T_e = K_t I_a$$

and

$$E_b = K_e \omega_m$$

the electrical and mechanical subsystems can be interconnected to obtain the complete motor model shown in Figure 2.1

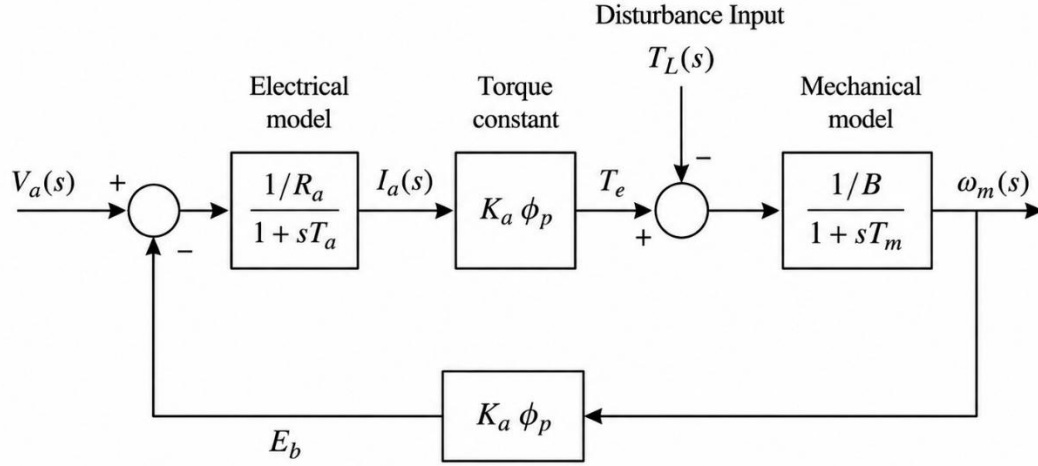


Figure 2.1: Control-Oriented Dynamic Model of the Separately Excited DC Motor

Taking the Laplace Transform of the complete model and applying the standard closed-loop relation $G(s)/(1 + G(s)H(s))$, the transfer function relating armature voltage to motor speed is obtained as

$$\frac{\omega_m(s)}{V_a(s)} = \frac{K_a \phi_p}{R_a B (1 + sT_a)(1 + sT_m) + (K_a \phi_p)^2}$$

The detailed derivation of the complete transfer function is provided in **Appendix A**.

2.4 PWM-Based Voltage Control

The mathematical model developed so far assumes that any desired armature voltage can be applied directly to the motor. In practice, however, the available DC supply voltage is fixed. Therefore, a method is required to generate different average armature voltages from a constant DC source.

This is achieved using a DC-DC chopper operating under Pulse Width Modulation (PWM).

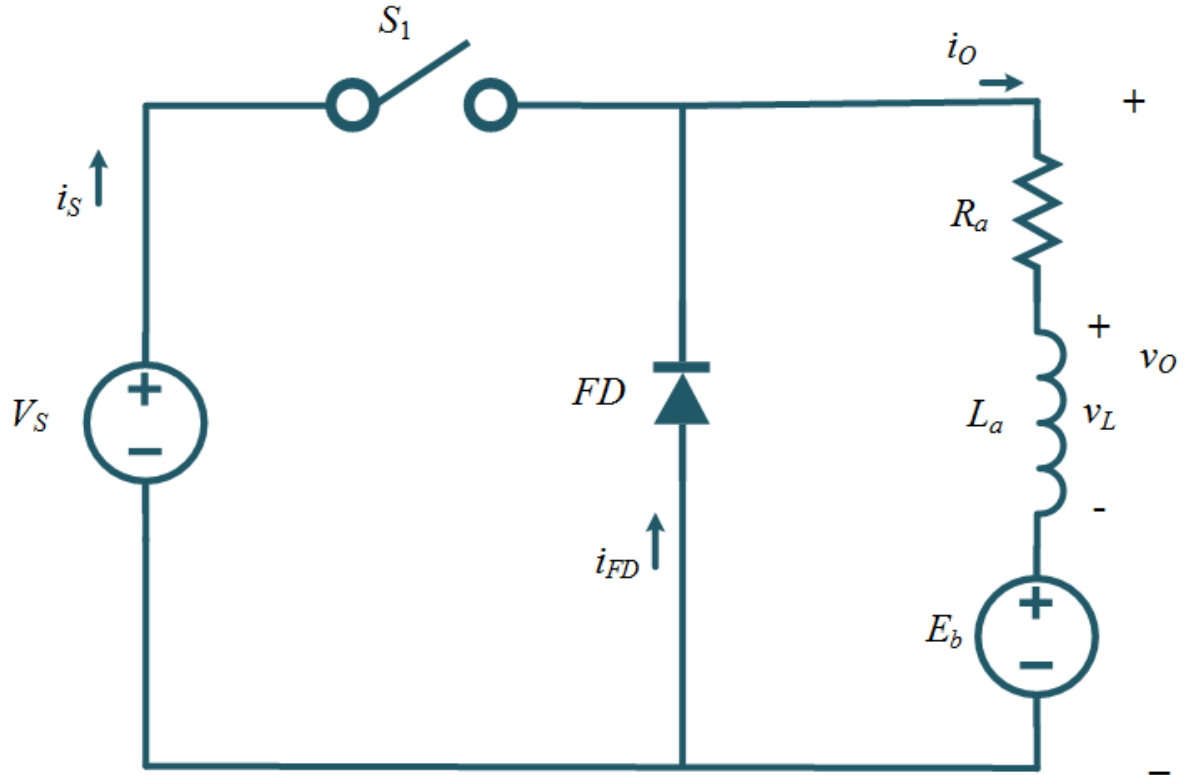


Figure 2.2: One-Quadrant Chopper Used for Armature Voltage Control

Source: *Electrical Workbook* [2]

The basic principle of PWM is straightforward. Instead of continuously varying the supply voltage, the power switch is rapidly turned ON and OFF. By varying the fraction of time for which the switch remains ON, the average voltage applied to the motor can be controlled.

The duty ratio is defined as

$$D = \frac{T_{ON}}{T_s}$$

where T_{ON} is the switch ON time and T_s is the switching period.

The average armature voltage becomes

$$V_a = DV_g$$

where V_g is the DC supply voltage.

For example, with a 24 V supply and a duty ratio of 0.5,

$$V_a = 0.5 \times 24 = 12V$$

Although the motor terminals experience a rapidly switching voltage waveform, the electrical inductance and mechanical inertia prevent the motor from

responding to the high-frequency switching components. Consequently, the motor primarily responds to the average value of the applied voltage.

PWM generation is implemented by comparing the controller output signal V_c with a high-frequency triangular carrier waveform V_m . Whenever the controller signal exceeds the carrier waveform, the switch is turned ON; otherwise it is turned OFF. The resulting pulse train controls the duty ratio and consequently the average armature voltage applied to the motor.

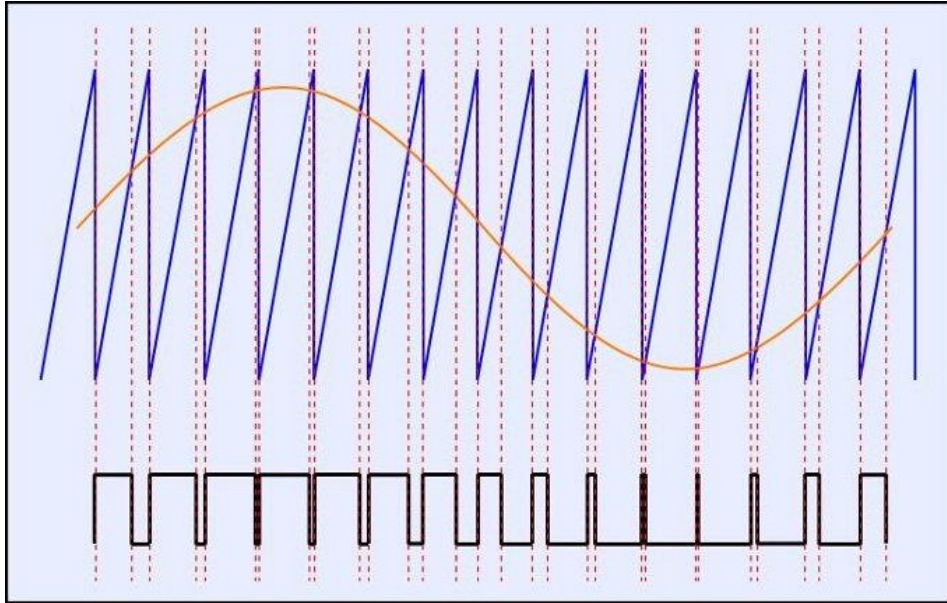


Figure 2.3: PWM generation through comparison of controller signal V_c and carrier waveform V_m

Source: MaxEmbedded [3]

The equivalent PWM gain may therefore be represented as

$$\frac{V_a}{V_c} = \frac{V_g}{V_m}$$

where V_c is the controller output signal and V_m is the carrier amplitude.

The derivation of the PWM gain model is provided in **Appendix B**.

2.5 Control-Oriented Plant Model

Combining the PWM converter model with the motor transfer function produces the complete control-oriented plant used for controller design.

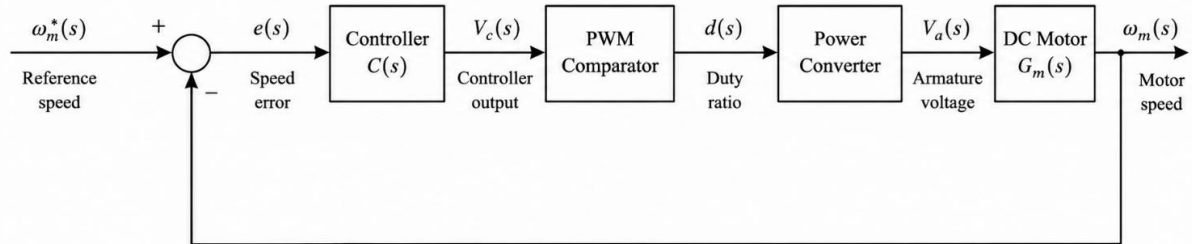


Figure 2.4: Control-Oriented Plant Model

The controller generates a control signal V_c , the PWM block converts this signal into an appropriate duty ratio, the chopper produces the required average armature voltage, and the motor responds by producing the desired speed.

With the mathematical model of the system now established, the next step is to design a controller capable of achieving the desired speed regulation and dynamic performance requirements.

3. Controller Design Using Frequency Domain Techniques

3.1 Design Objectives

Now that we have a mathematical model of our motor, the next question is straightforward: how do we design a controller?

From the previous chapter, our control-oriented plant consists of the motor transfer function together with the PWM gain. The only thing left to decide is the controller transfer function. Once we choose the controller, the complete behaviour of the system is determined.

So what exactly do we mean by "desired behaviour"?

There are several things we would like from our motor drive:

1. Settling Time (T_s) – How quickly should the motor reach the desired speed?
2. Steady-State Error (e_{ss}) – How accurately should the motor maintain the reference speed?
3. Stability – Will the system settle to the desired speed or oscillate around it?
4. Bandwidth – How rapidly can the motor respond to changing commands?
5. Attenuation – Should the controller react to high-frequency signals such as PWM switching ripple?

For example, suppose we continuously change the speed reference. Initially we change it slowly. The motor can easily follow these changes. Now we begin changing the reference faster and faster. Eventually the motor will no longer be able to keep up due to its electrical and mechanical dynamics.

This means the motor effectively responds only to a certain range of frequencies. Frequencies within this range are followed by the system, while higher frequencies are increasingly attenuated. This frequency range is characterized by the system bandwidth.

Similarly, consider our PWM switching frequency of 2 kHz. We do not want the controller reacting strongly to signals at such high frequencies. Neither the controller nor the motor should waste effort trying to follow switching ripple. Therefore high-frequency attenuation is also an important requirement.

So how do we actually design a controller that satisfies all these requirements?

One approach would be to keep changing controller parameters and repeatedly observe the step response. While this can work, it quickly becomes trial and error.

A more systematic approach is to analyse the system in the frequency domain.

Instead of asking:

"How does speed change with time?"

we ask:

"How does the system respond to signals of different frequencies?"

Low-frequency signals are followed well.

High-frequency signals are followed poorly.

Very high-frequency signals are strongly attenuated.

To study this behaviour, we use a Bode Plot.

A Bode plot consists of two graphs:

1. Magnitude Plot

2. Phase Plot

The magnitude plot tells us how strongly the system reacts to different frequencies, while the phase plot tells us how much phase lag is introduced by the system.

Using these plots, important quantities such as bandwidth, attenuation, gain margin and phase margin can be determined.

3.2 Loop Shaping

The objective of loop shaping is to modify the open-loop frequency response using a controller such that the resulting closed-loop system exhibits the desired performance characteristics.

Before proceeding with controller design, it is useful to understand how the performance requirements discussed earlier relate to the frequency response of a system. For a simple first-order linear system, quantities such as settling time, steady-state error and bandwidth can be directly related to system parameters. Similarly, stability can be quantified using gain margin and phase margin. These relationships allow time-domain performance requirements to be translated into frequency-domain design specifications.

The detailed derivations of the following relationships are provided in **Appendix C**.

Settling Time

For a first-order system,

$$T_s \approx \frac{4}{\omega_c}$$

where ω_c is the crossover frequency.

Thus, a higher crossover frequency results in a faster response.

Steady-State Error

For a unity feedback system,

$$e_{ss} = \lim_{s \rightarrow 0} \frac{1}{1 + G(s)H(s)}$$

Therefore, increasing the low-frequency loop gain reduces steady-state error.

Bandwidth

Bandwidth represents the range of frequencies that the system can effectively follow.

For a first-order system,

$$\omega_B \approx \omega_c$$

and therefore

$$T_s \approx \frac{4}{\omega_B}$$

showing the relationship between response speed and bandwidth.

Stability Margins

Stability is quantified using Gain Margin (GM) and Phase Margin (PM).

$$PM = 180^\circ + \angle G(j\omega_c)H(j\omega_c)$$

where ω_c is the gain crossover frequency.

Similarly,

$$GM = \frac{1}{|G(j\omega_{pc})H(j\omega_{pc})|}$$

where ω_{pc} is the phase crossover frequency.

These margins indicate how far the system is from instability.

Attenuation

The controller should respond strongly to low-frequency reference and disturbance signals while attenuating high-frequency components such as switching ripple and measurement noise.

Since the PWM converter operates at 2 kHz, sufficient attenuation is required near the switching frequency.

In addition to the above specifications, a desirable open-loop characteristic is to have a slope of approximately -20dB/dec around the crossover frequency. This causes the system to behave approximately as a first-order system near crossover and generally results in good stability margins and robust closed-loop performance.

3.3 Plant Used For Controller Design

The following design specifications were selected:

$$T_s \leq 0.1s$$

$$e_{ss} \leq 0.01$$

$$PM > 30^\circ$$

$$GM > 3\text{dB}$$

Attenuation at 1 kHz: 30dB

From

$$e_{ss} = \lim_{s \rightarrow 0} \frac{1}{1 + G(s)H(s)}$$

the required open-loop DC gain was found to be approximately 9999.

Similarly,

$$T_s = \frac{4}{\omega_c}$$

gives

$$\omega_c = 40; \text{rad/s}$$

The motor parameters used were:

Table 3.1: Parameters of the Separately Excited DC Motor

Parameter	Symbol	Value	Unit
Torque / Back EMF Constant	$K_a\phi$	0.0424	N·m/A ($\equiv$ V·s/rad)
Viscous Friction Coefficient	B	1.4×10^{-5}	N·m·s/rad
Moment of Inertia	J	8.473×10^{-6}	kg·m ²
Armature Resistance	R_a	4.62	Ω
Armature Inductance	L_a	4	mH
Mechanical Time Constant	T_m	0.6052	S
Electrical Time Constant	T_a	0.0008658	s

Substituting the motor parameters into the transfer function derived in Chapter 2 gives the motor transfer function:

$$\frac{23.21}{\left(\frac{s}{48.6482} + 1\right)\left(\frac{s}{1108.0327} + 1\right)}$$

The PWM gain was

$$\frac{V_g}{V_m} = \frac{24}{100} = 0.24$$

Therefore, the complete plant transfer function used for controller design becomes

$$\frac{5.5704}{\left(\frac{s}{48.6482} + 1\right)\left(\frac{s}{1108.0327} + 1\right)}$$

3.4 Controller Design by Loop Shaping

Now that the plant transfer function has been obtained, the next step is to design a controller that satisfies the performance specifications discussed previously.

Let us first examine the frequency response of the plant alone before adding any controller.

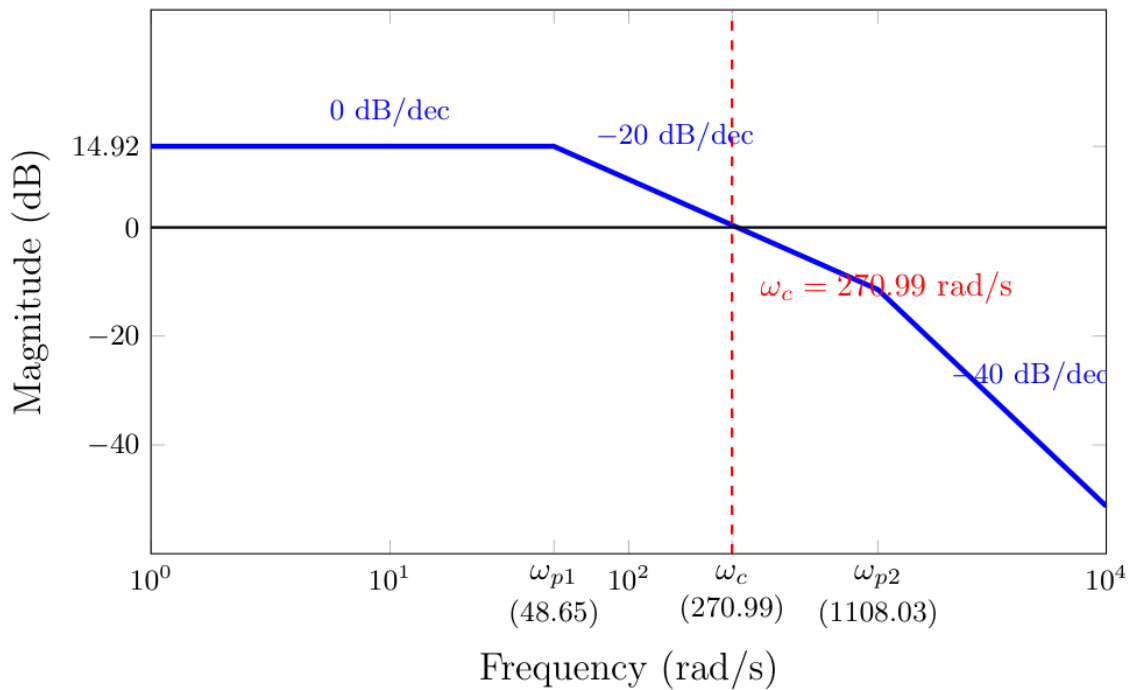


Figure 3.1 Magnitude Bode Plot of the Uncompensated Plant

From the Bode plot, the DC gain of the plant is approximately 14.92 dB.

However, from the steady-state error specification, we previously determined that an open-loop DC gain of approximately 9999 is required. In decibel form,

$$20 \log_{10}(9999) \approx 80 \text{ dB}$$

Therefore, additional gain must be added to the system. The required gain is calculated as

$$K = \frac{9999}{5.57} \approx 1795.2$$

Adding this gain shifts the magnitude plot upward.

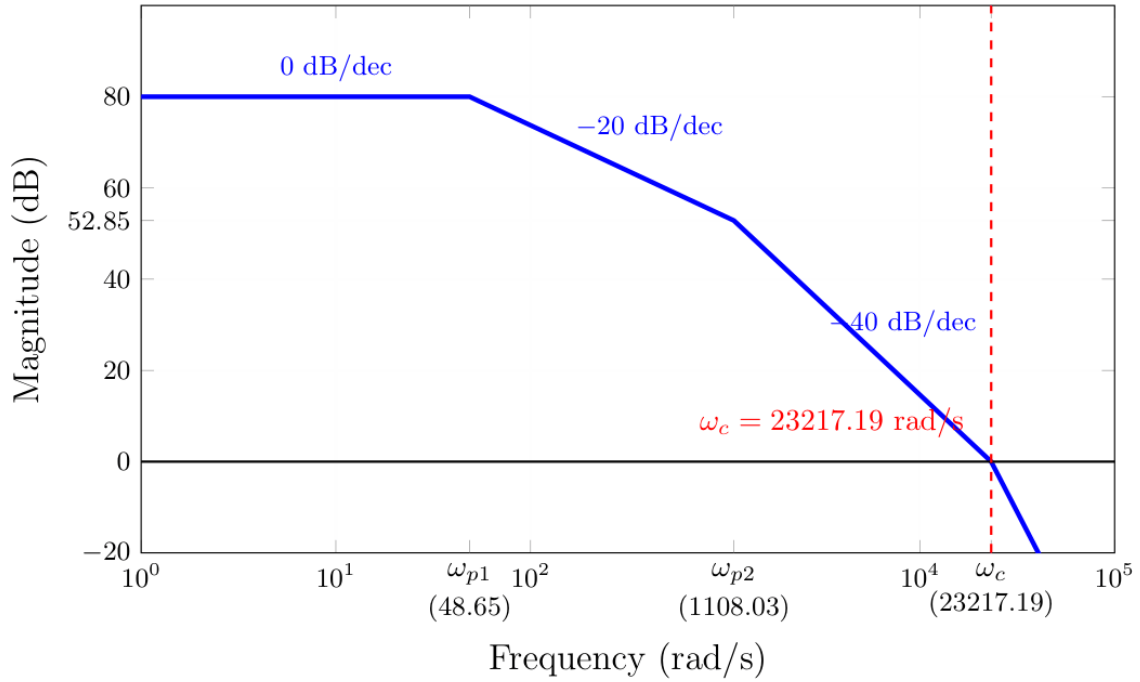


Figure 3.2: Bode Plot after Adding a Gain of Approximately 1795

Although the gain requirement is now satisfied, a new problem appears. The gain crossover frequency shifts to approximately

$$\omega_c = 23217 \text{ rad/s}$$

which is far greater than the desired crossover frequency.

A very high crossover frequency implies an unrealistically fast response and does not reflect the physical limitations of the motor. Since the motor mechanical dynamics are much slower than the electrical dynamics, it is desirable to place the crossover frequency at a more reasonable value, approximately 100 rad/s.

To achieve this, the magnitude plot must be forced to roll off before the mechanical pole frequency. Therefore, an additional pole is introduced at

$$\omega_{pc} = 0.002056 \text{ rad/s}$$

and a zero is introduced at

$$\omega_{zc} = 10 \text{ rad/s}$$

The zero also helps obtain an approximate slope of -20 dB/decade around crossover frequency, which generally leads to good stability characteristics.

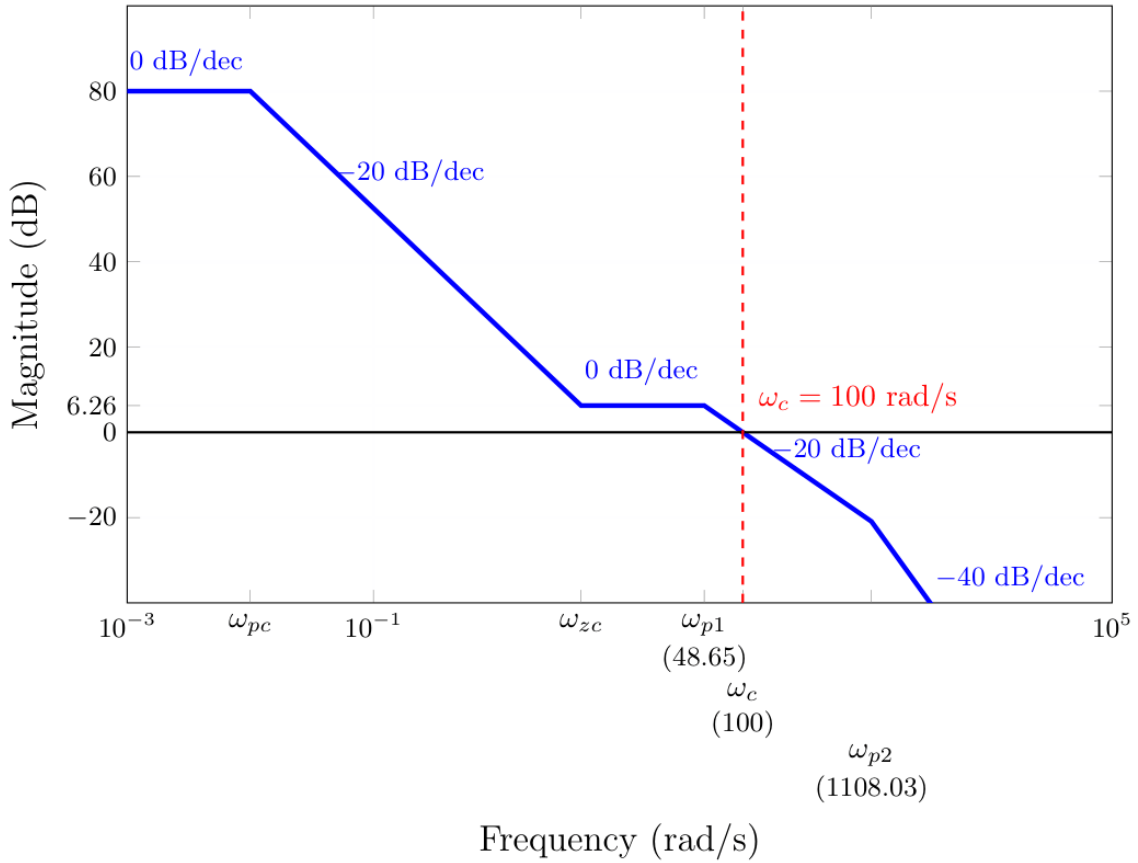


Figure 3.3: Bode Plot after Addition of Pole-Zero Compensation

The next step is to verify the stability of the compensated system. For this purpose, the phase response of the complete open-loop system is examined.

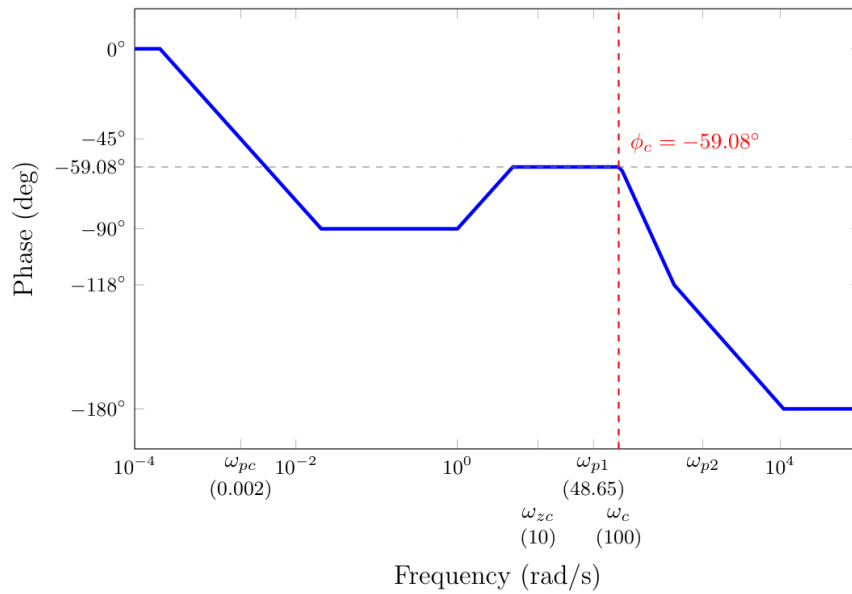


Figure 3.4: Phase Bode Plot of the Complete Open-Loop System

At the gain crossover frequency, the phase is approximately -59.08°

Therefore,

$$PM = 180^\circ - 59.08^\circ$$

$$PM = 120.92^\circ$$

Since the phase never crosses the 0° axis the gain margin is infinite.

The resulting controller transfer function is therefore

$$G_c(s) = \frac{1795.203217 \left(\frac{s}{10} + 1 \right)}{\left(\frac{s}{0.002056} + 1 \right)}$$

To verify the design, MATLAB was used to generate the exact Bode plot and closed-loop step response. The MATLAB code used for this analysis is provided in **Appendix D**.

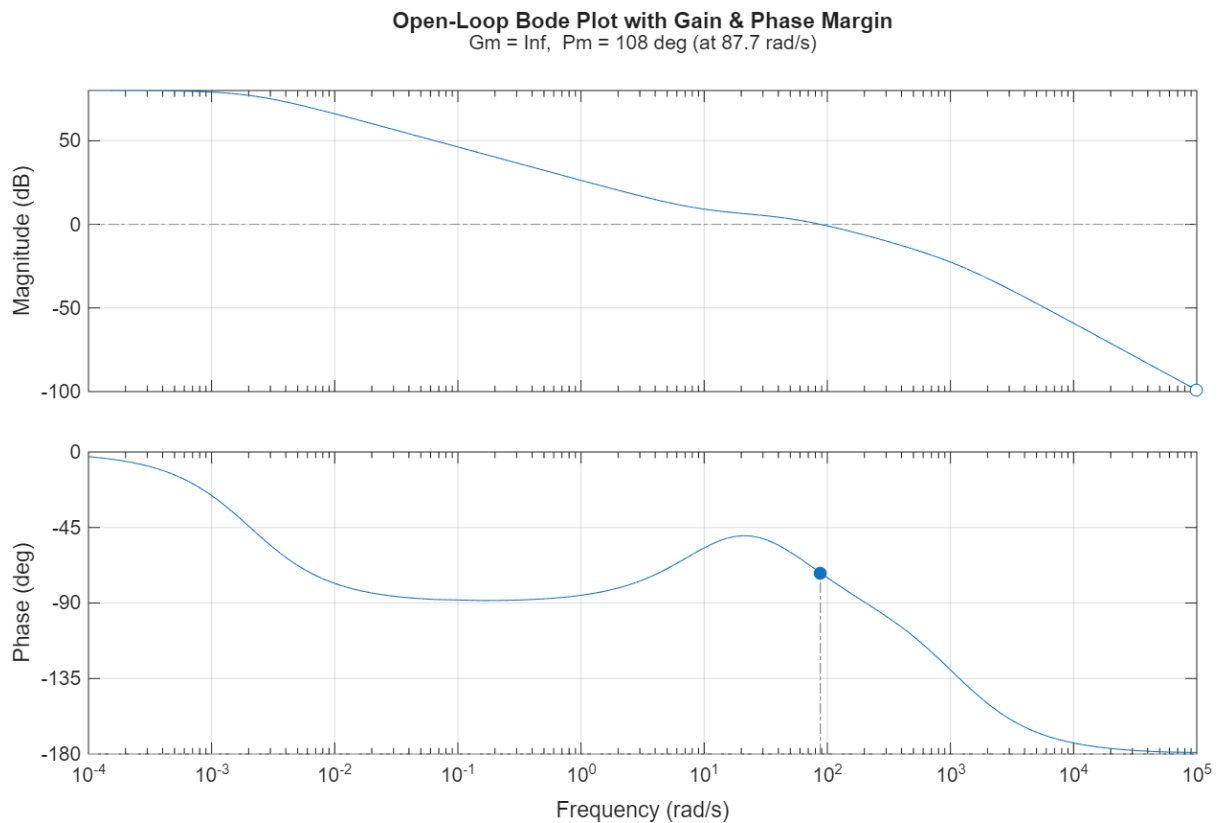


Figure 3.5: Exact Bode Plot of the Compensated System

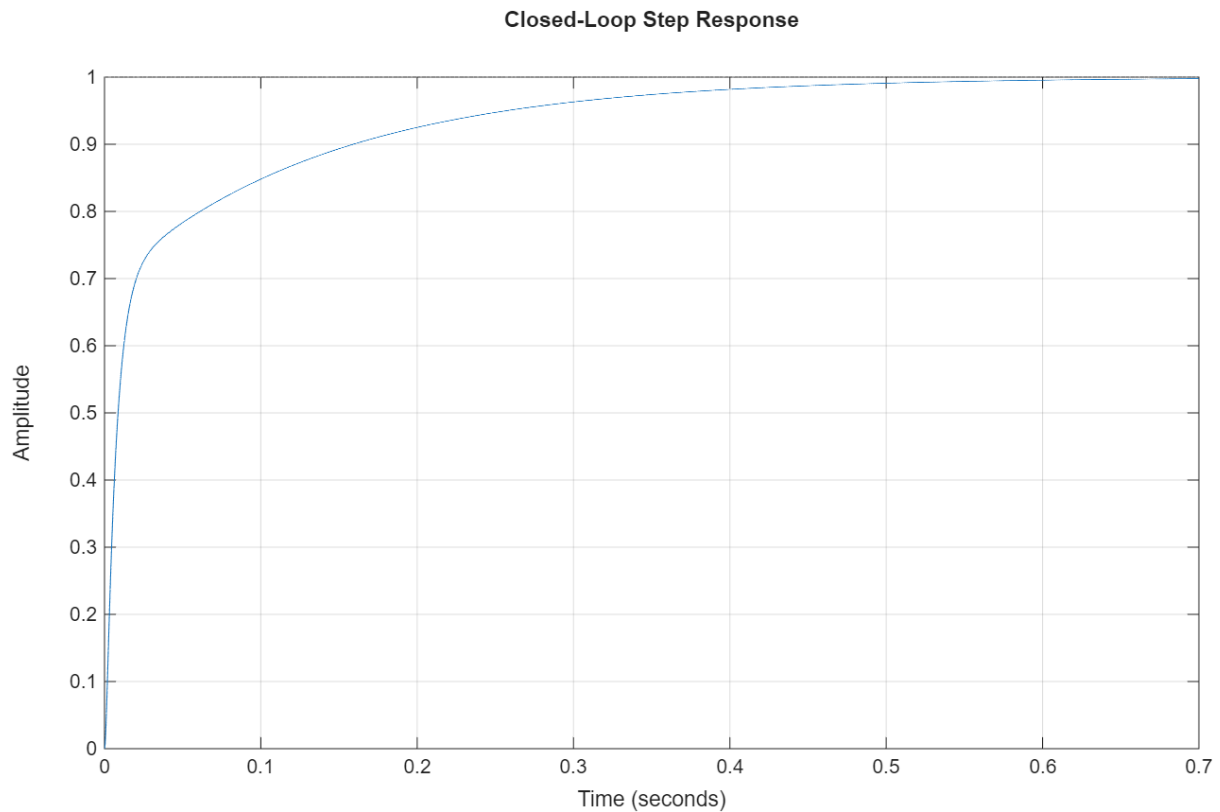


Figure 3.6: Closed-Loop Response of the Compensated System

From the exact Bode plot, the gain crossover frequency is approximately 87.7rad/s

and the phase margin is approximately 108°

The corresponding closed-loop response exhibits a settling time of $T_s = 0.3864\text{s}$

Although the crossover frequency of the compensated system was approximately 87.7 rad/s , the settling time relation $T_s \approx 4/\omega_c$ is only an approximation valid for systems operating within their linear region. The designed controller contained multiple poles and zeros, and therefore the exact settling time must be obtained from the closed-loop response.

3.5 Simulink Implementation of the Designed Controller

The controller obtained through frequency-domain loop shaping was next implemented in the complete motor drive model in Simulink. The objective of this step was to verify whether the performance predicted by the linear frequency-domain analysis could be achieved when the controller was connected to the actual motor model.

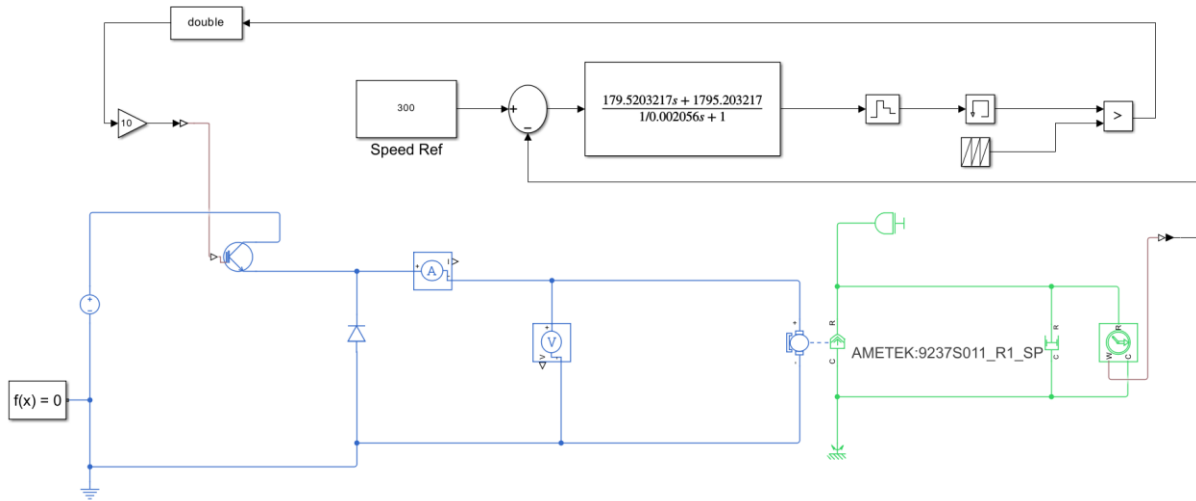


Figure 3.7: Simulink Implementation of the Designed Controller

The controller transfer function obtained from the loop-shaping procedure was used directly in the feedback loop. The controller output represents the desired armature voltage, which is applied to the motor model through the converter.

The resulting controller output and motor speed response are shown in Figure 3.8 and 3.9 respectively.

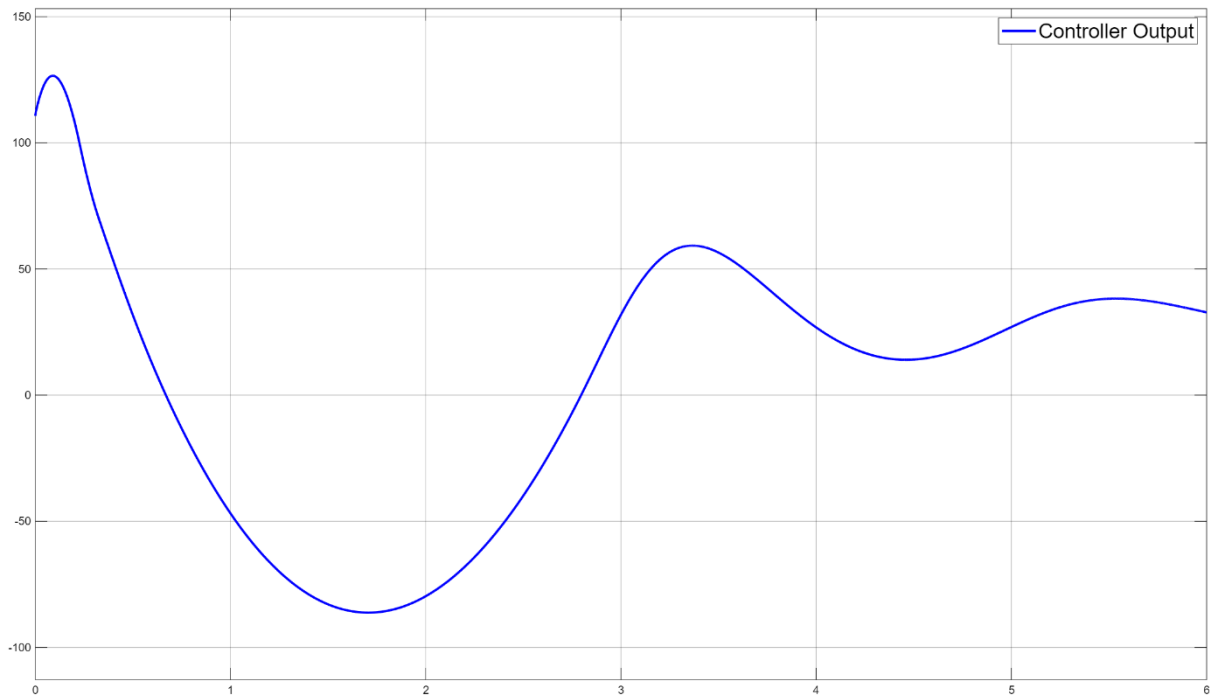


Figure 3.8: Controller Output Voltage Response

Figure 3.8 shows the controller output generated by the designed controller. During startup, the speed error is very large and therefore the controller demands a correspondingly large control action.

It can be observed that the controller output reaches values well above the operating range required for PWM generation. Since the PWM modulator can only produce duty ratios within its allowable limits, a significant portion of the controller output lies outside the physically realizable operating region.

This indicates that the controller is demanding a control effort greater than what can actually be applied by the converter. In practice, the controller output must therefore be limited before being sent to the PWM stage.

At this stage, we simply note that the controller output exceeds the allowable range. The effect of this limitation on the overall system response will become evident when the motor speed response is examined.

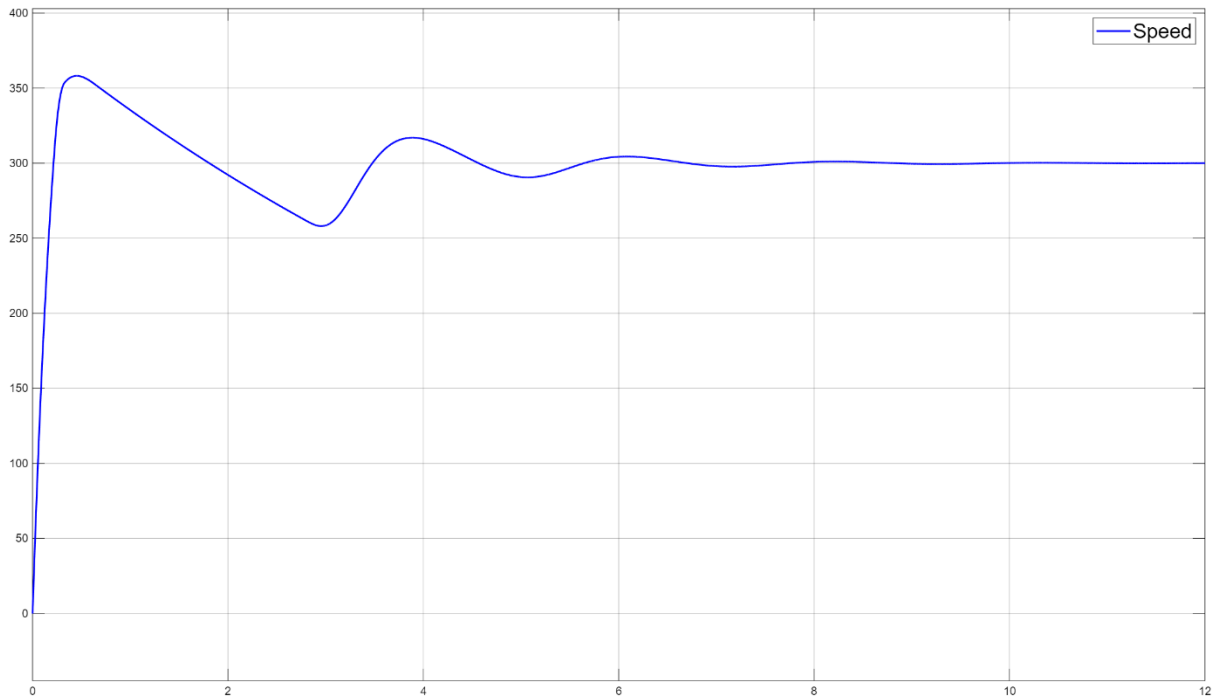


Figure 3.9: Closed-Loop Speed Response of the Motor

Table 3.2: Performance of the Designed Controller in Simulink

Peak Speed	358.1783 rad/s
Overshoot	19.3928 %
Steady-State Speed	299.9821 rad/s
Steady-State Error	0.005973 %
Settling Time	5.3661 s

From Figure 3.9 and Table 3.2, it can be observed that the motor successfully reaches the desired speed and the steady-state error remains very small. However, the settling time obtained from simulation is approximately 5.37 s, which is significantly larger than the value predicted during the frequency-domain design stage.

The frequency-domain analysis predicted a settling time of approximately 0.386 s based on the crossover frequency and linear system assumptions. However, the controller output shown previously exceeds the allowable operating range of the PWM stage. Consequently, the controller cannot apply the full control action demanded by the design.

This causes the system to operate under saturation conditions during a significant portion of the transient response. Once saturation occurs, the system no longer behaves according to the linear model used during loop-shaping analysis. As a

result, the actual response becomes considerably slower than the response predicted from the Bode plot and crossover frequency.

Although the controller satisfies the steady-state error requirement, the dynamic performance remains unsatisfactory. Therefore, the controller structure will be further improved by introducing integral action in order to enhance performance and investigate whether a PI controller can provide a better practical response.

3.6 PI Controller Design

In the previous section, we designed a controller using loop-shaping techniques and verified its performance in Simulink. Although the controller was able to regulate the motor speed, the practical response was significantly slower than predicted by the frequency-domain analysis.

Another observation is that our controller contains only a gain together with a pole-zero pair. Although the steady-state error requirement was satisfied, it is often desirable to include integral action in the controller structure as integrators help eliminate steady-state error and improve low-frequency tracking performance.

Therefore, let us now design a PI controller and compare its performance with the previously designed controller.

The transfer function of a PI controller is given by

$$G_c(s) = K_p + \frac{K_i}{s}$$

or equivalently

$$G_c(s) = \frac{K_p s + K_i}{s}$$

Using the same loop-shaping procedure discussed previously, the resulting PI controller was obtained as

$$G_c(s) = \frac{3.69 \left(\frac{s}{10} + 1 \right)}{s}$$

The Bode plot and closed-loop step response of the compensated system is shown below.

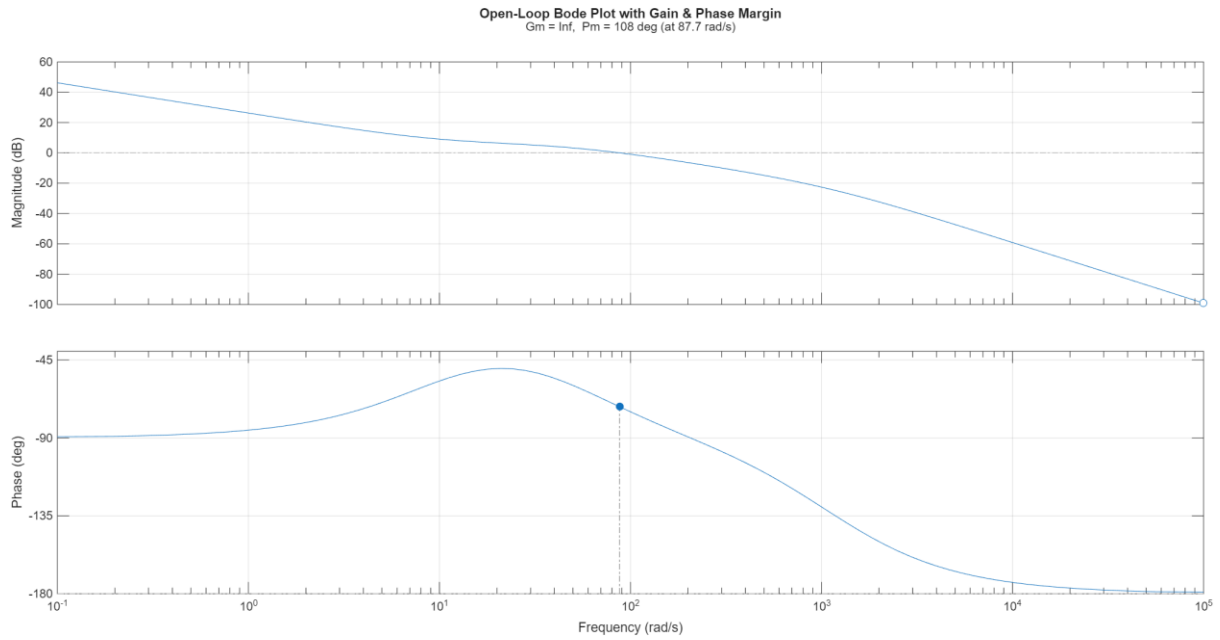


Figure 3.10: Bode Plot of the System with PI Controller

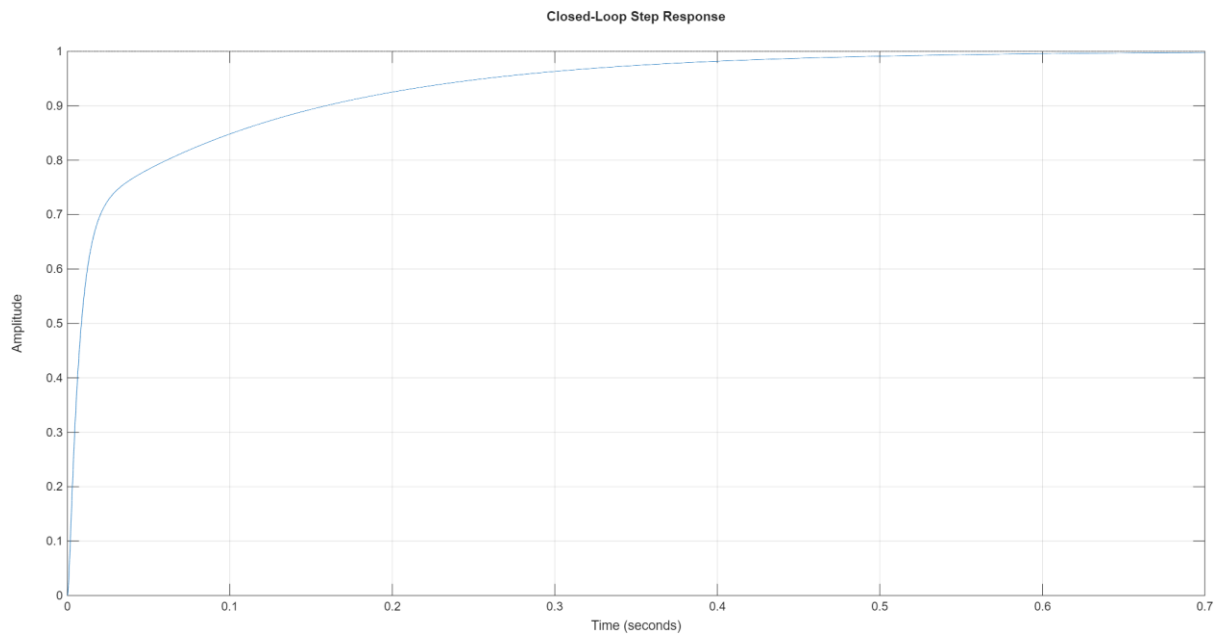


Figure 3.11: Closed-Loop Speed Response of the Motor

From the Bode plot, the predicted settling time was found to be

$$T_s = 0.3865s$$

which is nearly identical to the value obtained using the previous controller.

The gain margin, phase margin and crossover frequency also remained nearly unchanged. This indicates that the overall closed-loop behaviour predicted by frequency-domain analysis should be very similar for both controllers.

To verify this, the controller was implemented in Simulink and the corresponding speed response was observed.

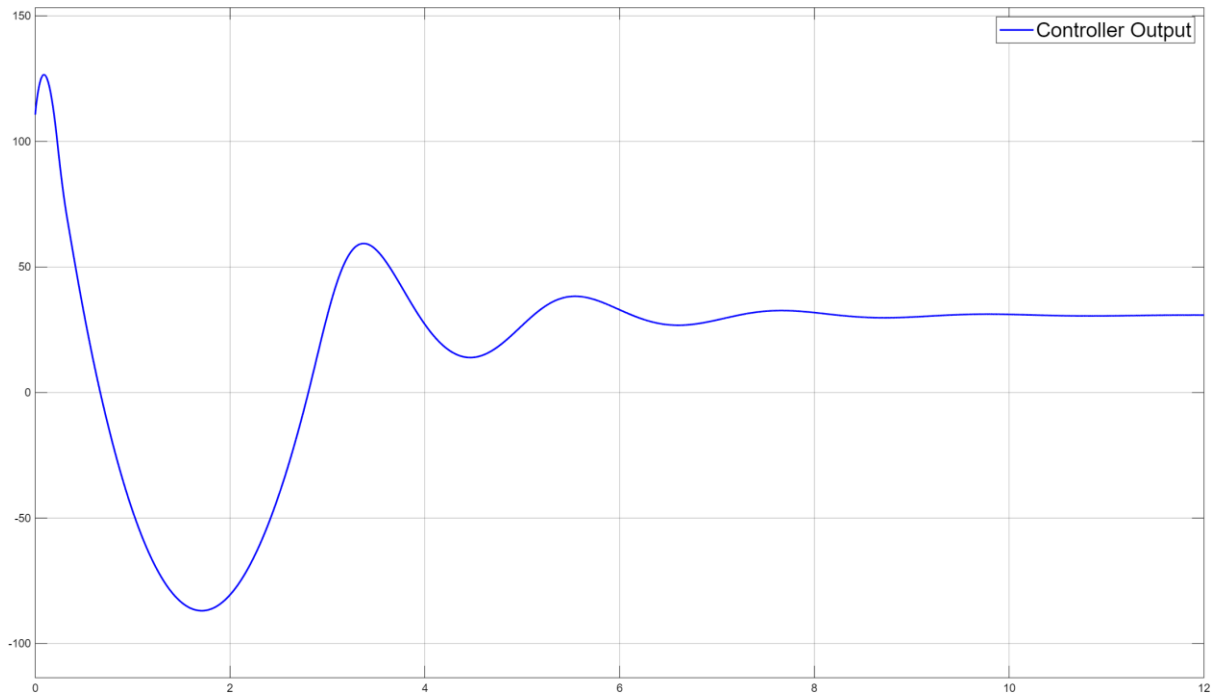


Figure 3.12: PI Controller Output Voltage Response

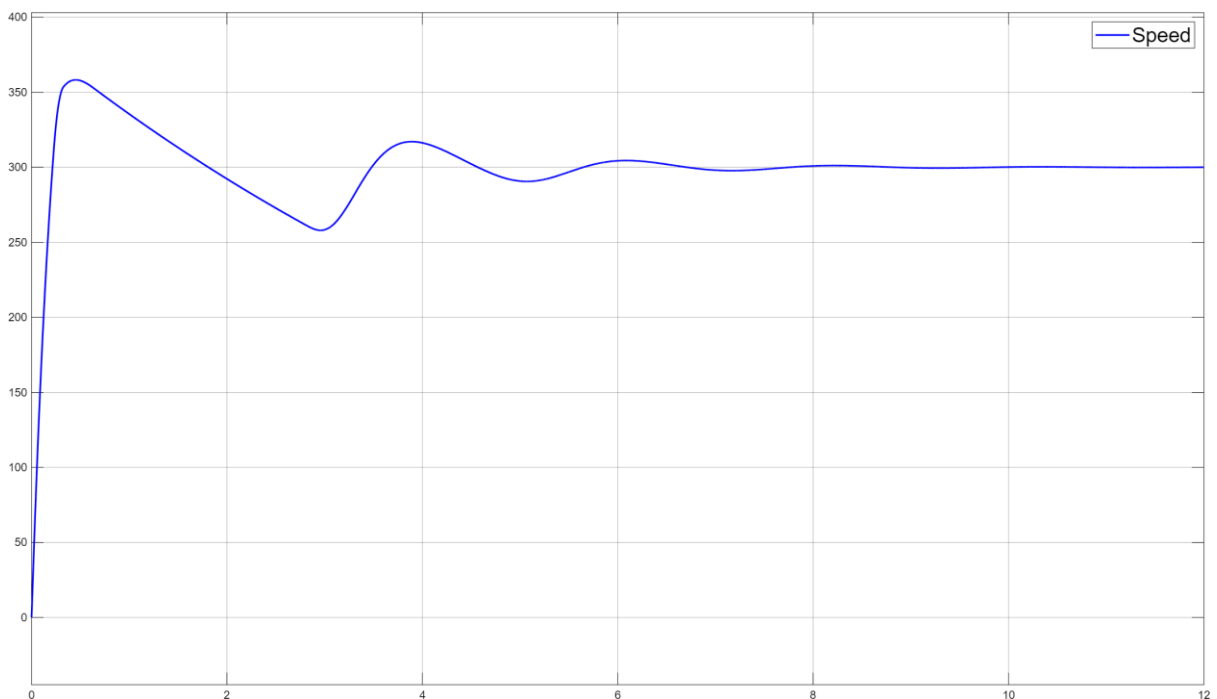


Figure 3.13: Closed-Loop Response using PI Controller

The performance obtained from simulation is summarized in Table 3.3.

Table 3.3: Performance of the PI Controller in Simulink

Peak Speed	358.2332 rad/s
Overshoot	19.4111 %
Steady-State Speed	299.9980 rad/s
Steady-State Error	0.000674 %
Settling Time	5.3701 s

Comparing these results with those obtained using the previous controller, it can be observed that the responses are nearly identical. The overshoot, settling time and steady-state performance show only minor differences.

This behaviour is consistent with the frequency-domain analysis, which predicted very similar gain margins, phase margins and crossover frequencies for both controllers.

Therefore, although the introduction of integral action improved the theoretical controller structure, it did not resolve the primary issue responsible for the poor practical response. The settling time remained significantly larger than the value predicted during controller design.

This suggests that the dominant limitation is not the controller structure itself, but rather practical implementation constraints that were not included in the linear frequency-domain model. These limitations will be investigated and addressed in the following chapter.

4. Practical Controller Implementation and Performance Improvements

4.1 Limitations of the Initial Controller Design

In the previous chapter, two controllers were designed using frequency-domain techniques and verified through simulation. Both controllers satisfied the steady-state error requirements and exhibited acceptable stability margins. However, the practical response obtained from Simulink was significantly slower than the response predicted by the frequency-domain analysis.

The frequency-domain design predicted a settling time of approximately 0.386 s. However, when implemented in the complete motor drive model, the settling time increased to approximately 5.37 s.

So why did this happen?

The answer can be understood by examining the controller output. During startup, the speed error is very large because the motor is initially at rest while the reference speed is 300 rad/s. Consequently, the controller attempts to generate a very large control signal in order to rapidly eliminate the error.

However, the PWM stage can only operate within a finite range. In the one-quadrant converter, the duty ratio must satisfy

$$0 \leq d \leq 1$$

Therefore, any controller output corresponding to a duty ratio greater than 100% cannot be physically realized. Similarly, negative duty ratios are not possible in a one-quadrant converter.

As a result, whenever the controller demands an output beyond these limits, the converter enters saturation.

During saturation, increasing the controller output further has no effect on the converter output. The converter is already operating at its maximum allowable value. Consequently, the controller temporarily loses its ability to regulate the motor voltage.

This behaviour explains why the practical response differs significantly from the response predicted by the linear frequency-domain model. The Bode plot assumes a linear system, whereas saturation introduces a nonlinear element into the control loop.

Therefore, before attempting further performance improvements, it is necessary to address the effects of saturation.

4.2 Integrator Windup and Anti-Windup Compensation

The effects of saturation become particularly severe when integral action is present in the controller.

To understand why this happens, consider a simple example.

Suppose a drone is required to reach an altitude of 50 m. Initially the drone is on the ground and therefore the altitude error is 50 m.

Now suppose a heavy payload is attached to the drone and the maximum rotor speed available is 1000 rpm.

Since the error remains large, the integral controller continuously accumulates error. By the time the rotor reaches its physical limit of 1000 rpm, the integrator may already be demanding a much larger speed, perhaps 1500 rpm or even 2000 rpm.

The important point is that the drone cannot actually follow these commands because it is already operating at its physical limit.

However, the integrator has no knowledge of this limitation. It continues accumulating error and continues increasing its output.

As the drone begins climbing, the error gradually decreases. Nevertheless, the integrator has already accumulated a very large stored value.

Even when the drone reaches the desired altitude, the controller may still be demanding excessive rotor speed due to the accumulated integral action. Consequently, the drone overshoots its target and takes a long time to settle.

This phenomenon is known as Integrator Windup.

A very similar situation occurs in motor drives. Whenever the controller demands voltages beyond the converter capability, the converter saturates. However, the integrator continues accumulating error, resulting in excessive overshoot and increased settling time.

To overcome this problem, an Anti-Windup mechanism was introduced.

The basic idea is simple.

First, the controller output is compared with the saturated output actually applied to the converter. If the two values differ, saturation has occurred.

Next, the sign of the controller output and the sign of the error are examined.

If both signs indicate that the integrator is continuing to drive the controller deeper into saturation, the integral path is temporarily disabled.

In other words, the controller behaves as a proportional controller during the saturation period.

Once the controller output returns to the allowable operating range, the integral path is re-enabled and normal PI operation resumes.

A saturation block was also introduced directly after the controller to ensure that only physically realizable commands are applied to the converter.

The resulting anti-windup implementation is shown below.

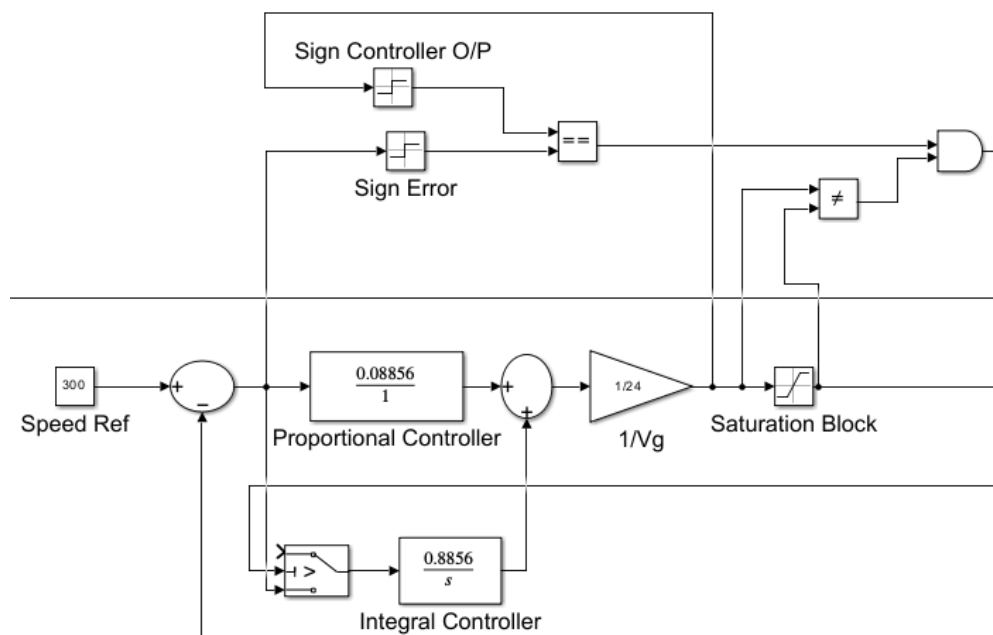


Figure 4.3: Anti-Windup Implementation using Integrator Clamping

This modification prevents excessive accumulation of integral action and significantly improves transient performance.

4.3 Duty-Ratio Based Controller Implementation

Up to this point, the controller output represented a desired armature voltage.

However, in practice the converter does not directly accept a voltage command. Instead, it operates by controlling the PWM duty ratio.

Therefore, rather than commanding voltage directly, the controller was modified to generate a duty-ratio command.

Since the converter supply voltage is

$$V_g = 24V$$

the required duty ratio can be obtained from

$$d = \frac{V_c}{24}$$

where V_c is the controller output voltage.

A saturation block was then introduced to ensure that

$$0 \leq d \leq 1$$

Thus, all duty-ratio commands remain physically realizable.

After introducing the division by 24 V, the PWM gain changed.

Previously, the PWM and converter combination contributed the gain

$$\frac{V_g}{V_m} = \frac{24}{100} = 0.24$$

However, after converting the controller output directly into duty ratio, the factor V_g is effectively cancelled and the PWM gain becomes unity.

To preserve the overall loop gain, the controller gain was therefore modified accordingly.

The resulting controller became

$$G_c(s) = \frac{0.8856 \left(\frac{s}{10} + 1 \right)}{s}$$

where

$$0.8856 = 3.69 \times 0.24$$

The modified controller together with anti-windup compensation was then implemented and tested.

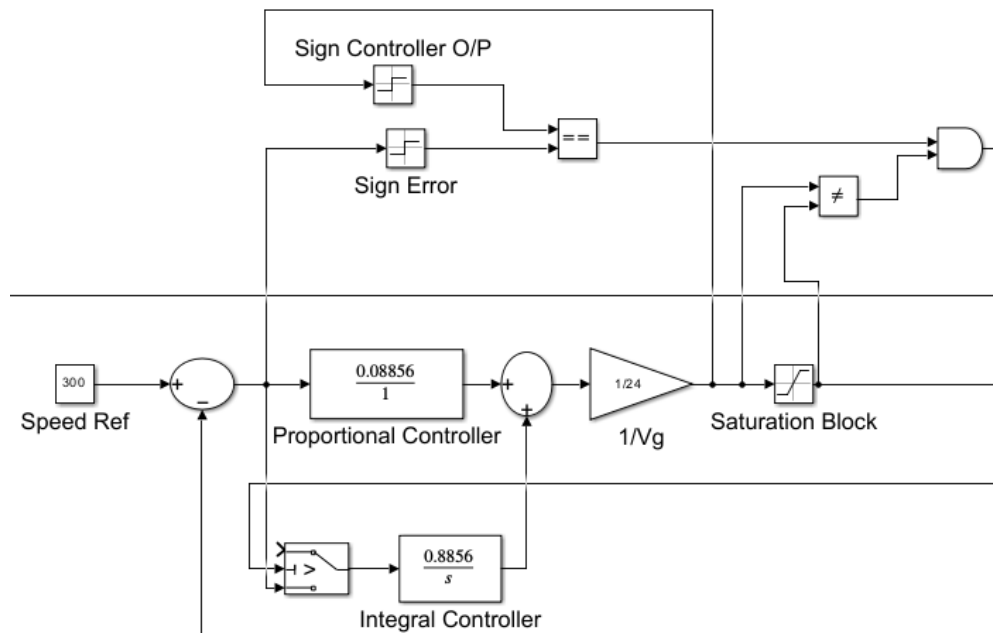


Figure 4.4: Duty-Ratio Based Controller Implementation

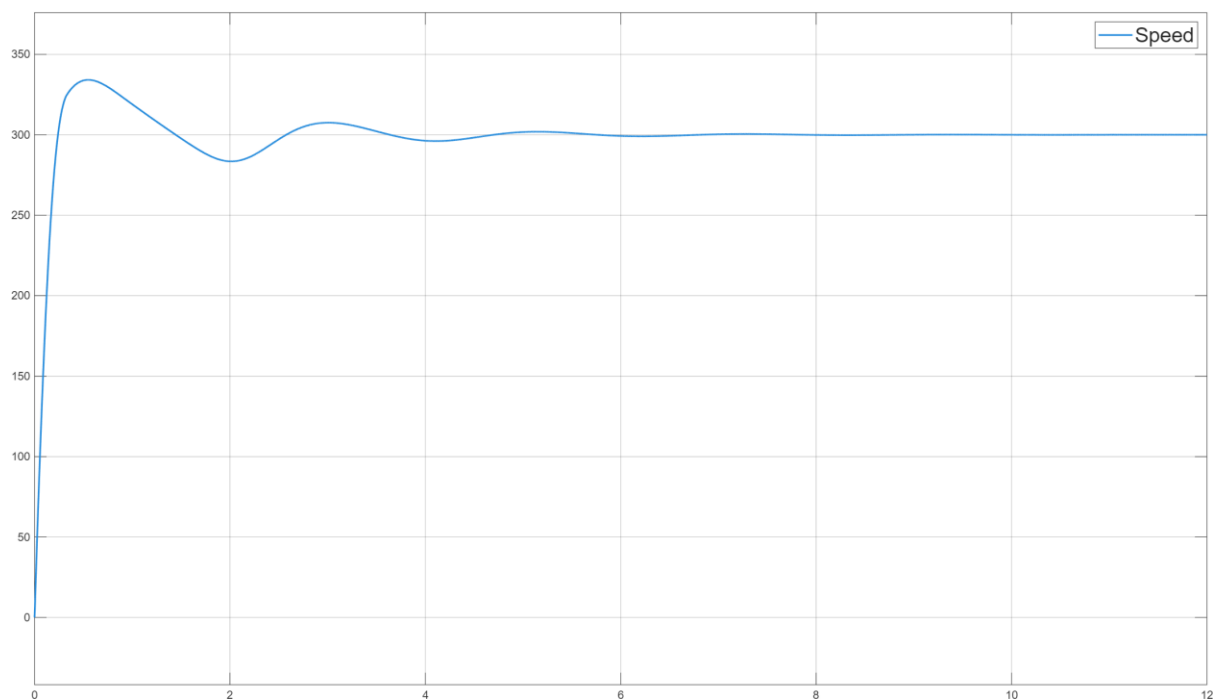


Figure 4.5: Speed Response after Anti-Windup and Duty-Ratio Implementation

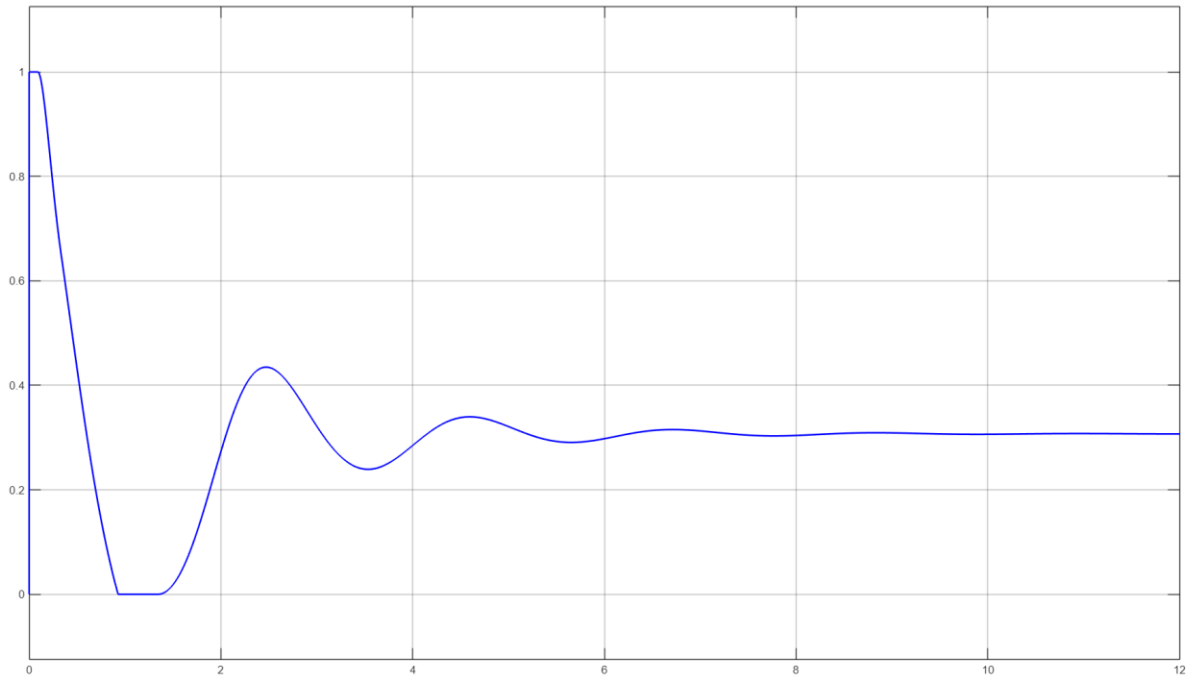


Figure 4.6: Controller Output after Anti-Windup and Duty-Ratio Implementation

The resulting performance is summarized in Table 4.1.

Table 4.1: Performance after Anti-Windup and Duty-Ratio Implementation

Peak Speed	334.1434 rad/s
Overshoot	11.3811 %
Steady-State Speed	300.0039 rad/s
Steady-State Error	0.001309 %
Settling Time	3.2392 s

Comparing these results with those obtained in Chapter 3, a substantial improvement can be observed. The settling time decreased from approximately 5.37 s to 3.24 s, while overshoot was reduced from approximately 19.4% to 11.4%.

Although the response is now significantly better, the performance is still slower than desired. Therefore, further improvements are required.

One important observation made during analysis of the motor model is that the electrical dynamics are considerably faster than the mechanical dynamics. This naturally motivates the use of separate current and speed control loops.

The next section therefore introduces cascaded speed-current control.

4.4 Cascaded Speed-Current Control

Although anti-windup compensation and duty-ratio based implementation significantly improved performance, the response was still slower than desired.

To understand how further improvements can be achieved, let us revisit the motor model developed in Chapter 2.

The DC motor contains two distinct dynamic subsystems:

1. Electrical Dynamics
2. Mechanical Dynamics

The electrical dynamics determine how quickly the armature current responds to voltage changes, while the mechanical dynamics determine how quickly the motor speed responds to torque changes.

From the motor parameters obtained previously,

$$T_a = 0.0008658 \text{ s}$$

and

$$T_m = 0.6052 \text{ s}$$

Clearly,

$$T_a \ll T_m$$

which means the electrical subsystem is significantly faster than the mechanical subsystem.

This observation suggests that attempting to control both current and speed using a single controller may not be the most effective approach.

Instead, it is advantageous to control the fast electrical dynamics and slow mechanical dynamics separately.

Therefore, a Cascaded Speed-Current Control structure was introduced.

In this architecture, the outer speed controller no longer directly generates the motor voltage command.

Instead, the speed controller generates a Current Reference.

$$I_{ref}$$

The current controller then compares this reference current with the actual motor current and generates the required voltage command.

The voltage command is subsequently converted into duty ratio and applied to the PWM converter.

Thus, the control hierarchy becomes

$$\text{Speed} \rightarrow \text{Current} \rightarrow \text{Voltage}$$

rather than directly

$$\text{Speed} \rightarrow \text{Voltage}$$

The inner current loop rapidly regulates motor torque while the outer speed loop regulates motor speed.

Since motor torque is directly proportional to armature current,

$$T_e = K_t I_a$$

accurate current control also results in accurate torque control.

This cascaded structure is widely used in industrial motor drives because it provides improved dynamic performance, better disturbance rejection and more predictable controller design.

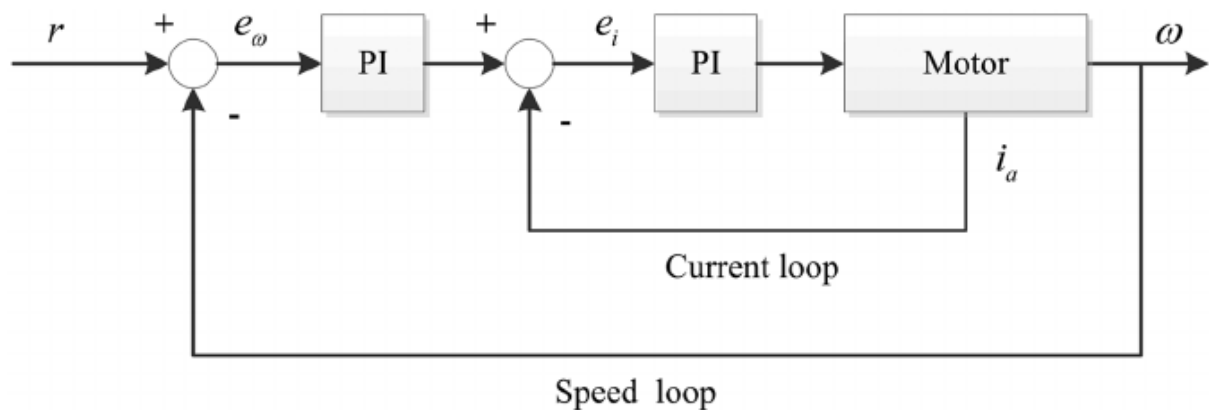


Figure 4.7: Cascaded Speed-Current Control Structure

Source: Electronics Stack Exchange [4]

The next step is therefore to design the inner current controller.

4.5 Current Controller Design

Since the current controller operates on the electrical subsystem, the electrical dynamics of the motor must first be examined.

The armature voltage equation is

$$V_a = R_a I_a + L_a \frac{dI_a}{dt} + E_b$$

where

$$E_b = K_e \omega$$

represents the motor back EMF.

During current-loop design, the electrical dynamics are much faster than the mechanical dynamics. Therefore the motor speed changes very slowly compared to the armature current.

Consequently, the back EMF term can be treated as a disturbance and neglected for controller design purposes.

The current dynamics therefore become

$$V_a = R_a I_a + L_a \frac{dI_a}{dt}$$

Taking the Laplace Transform,

$$\frac{I_a(s)}{V_a(s)} = \frac{1}{L_a s + R_a}$$

This is the transfer function of the electrical subsystem.

Rearranging into standard first-order form,

$$G_i(s) = \frac{\frac{1}{R_a}}{\left(\frac{s}{\frac{1}{T_a}}\right) + 1}$$

Substituting the motor parameters gives

$$G_i(s) = \frac{0.2165}{(s/1155) + 1}$$

The electrical time constant was previously found to be

$$T_a = 0.0008658 \text{ s}$$

which confirms that the electrical subsystem is extremely fast.

The PWM switching frequency used in this project was
 $f_{sw} = 2000Hz$

A commonly used guideline in control design is that the controller bandwidth should be approximately one-tenth of the switching frequency.

Therefore,

$$f_c \approx \frac{f_{sw}}{10}$$

giving

$$f_c \approx 200Hz$$

or

$$\omega_c = 2\pi f_c$$

$$\omega_c \approx 1256rad/s$$

A PI controller was selected because it provides zero steady-state current error while maintaining fast current tracking.

The controller structure is

$$G_c(s) = K_p + \frac{K_i}{s}$$

Initially,

$$K_p = 5$$

and

$$K_i = 50$$

which gives a controller zero at

$$\omega_z = \frac{K_i}{K_p}$$

$$\omega_z = 10rad/s$$

However, this zero is located far below the electrical pole at approximately
 $1155 rad/s$

To improve loop-shaping characteristics, the controller zero was shifted closer to the motor electrical pole.

Therefore,

$$\omega_z \approx 1000 \text{ rad/s}$$

was selected.

Using

$$K_i = K_p \omega_z$$

gives

$$K_i = 5 \times 1000 = 5000$$

Thus, the final current controller parameters became

$$K_p = 5$$

$$K_i = 5000$$

The resulting current controller provides high-bandwidth current regulation, zero steady-state current error and significantly faster dynamics than the outer speed loop.

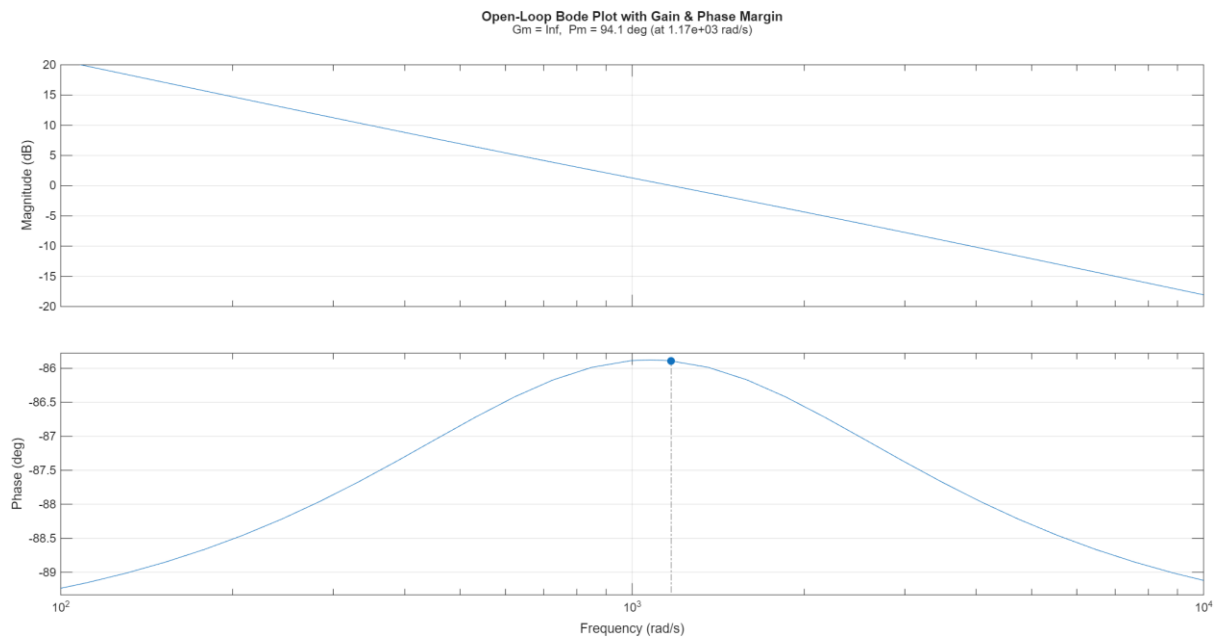


Figure 4.8: Current-Loop Bode Plot

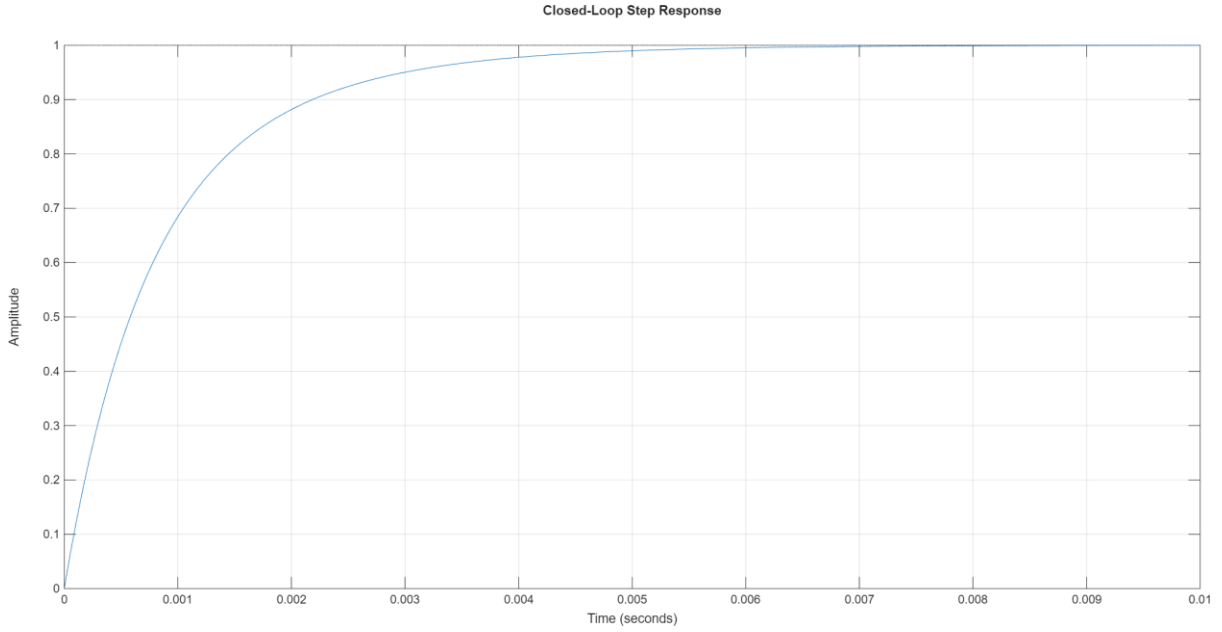


Figure 4.9: Current-Loop Step Response

The resulting controller achieved a crossover frequency of approximately 1170 rad/s while maintaining adequate stability margins.

4.6 Retaining the Existing Speed Controller

After designing the inner current loop, the next question is whether the outer speed controller should also be redesigned.

Ideally, once the current loop has been closed, the speed controller should be designed using only the mechanical dynamics

$$\frac{\omega(s)}{I_a(s)} = \frac{K_t}{Js + B}$$

because the electrical subsystem is now regulated by the current controller.

However, before redesigning the speed controller, it is useful to examine the relative bandwidths of the two control loops.

The current-loop crossover frequency is approximately

$$\omega_{c,current} = 1170 \text{ rad/s}$$

while the previously designed speed controller has a crossover frequency of approximately

$$\omega_{c,speed} = 88 \text{ rad/s}$$

Therefore,

$$\frac{\omega_{c,current}}{\omega_{c,speed}} = \frac{1170}{88} \approx 13$$

This means that the current loop is approximately thirteen times faster than the speed loop.

From the viewpoint of the speed controller, the current loop therefore behaves almost instantaneously.

Whenever the speed controller demands a current reference, the current controller rapidly forces the motor current to track that reference.

An intuitive way to understand this is that the speed controller effectively sees an ideal torque source.

By the time the motor speed begins changing appreciably, the current has already reached its commanded value.

Therefore, the previously designed speed controller was retained and validated through simulation instead of being redesigned.

Simulation results confirmed that the existing speed controller remained suitable when combined with the high-bandwidth current loop.

4.7 Current Limiting and Anti-Windup Implementation

Although the cascaded control structure provides significantly improved performance, practical operating limits must still be considered.

The output of the speed controller becomes the current reference for the inner current loop. Therefore, large speed errors during startup or disturbance conditions can cause the speed controller to demand unrealistically large currents.

In theory, increasing current increases torque and therefore improves acceleration. However, real motors and power converters possess finite current ratings. Excessive current results in increased copper losses, excessive heating and possible damage to both the motor and converter.

Therefore, a current limiting mechanism was introduced.

The rated motor current is

$$I_{rated} = 2.4 \text{ A}$$

To permit temporary overload capability while maintaining safe operation,

$$I_{max} = 1.2 I_{rated}$$

giving

$$I_{max} = 2.88 \text{ A}$$

which was rounded to

$$I_{max} = 3 \text{ A}$$

Therefore, the speed controller output was limited to

$$0 \text{ A} \leq I_{ref} \leq 3 \text{ A}$$

This ensures that current demand always remains within safe operating limits.

The anti-windup mechanism used previously in the speed controller was based on integrator clamping. However, after introducing the cascaded architecture, a different anti-windup approach was implemented for the current controller.

Unlike the speed controller, the current controller output is ultimately converted into a duty ratio before being applied to the PWM stage. Since the PWM converter can only operate within

$$0 \leq d \leq 1$$

the duty command is subjected to saturation before being applied to the converter.

Whenever the current controller demands a duty ratio outside this range, the converter cannot provide the requested control effort. However, the integral term of the current controller continues integrating the error, causing integrator windup.

To overcome this problem, a Back-Calculation Anti-Windup scheme was implemented.

The unsaturated duty command is compared with the saturated duty command actually applied to the PWM stage. The difference between these two signals represents the amount by which the controller output exceeds the converter capability.

The anti-windup correction signal is given by

$$e_{aw} = e + K_{aw}(d_{sat} - d)$$

where d is the unsaturated duty command and d_{sat} is the saturated duty ratio applied to the converter.

The anti-windup gain was selected as

$$K_{aw} = \frac{K_i}{K_p}$$

For the designed current controller,

$$K_p = 5$$

and

$$K_i = 5000$$

therefore

$$K_{aw} = 1000$$

The resulting implementation is shown below.

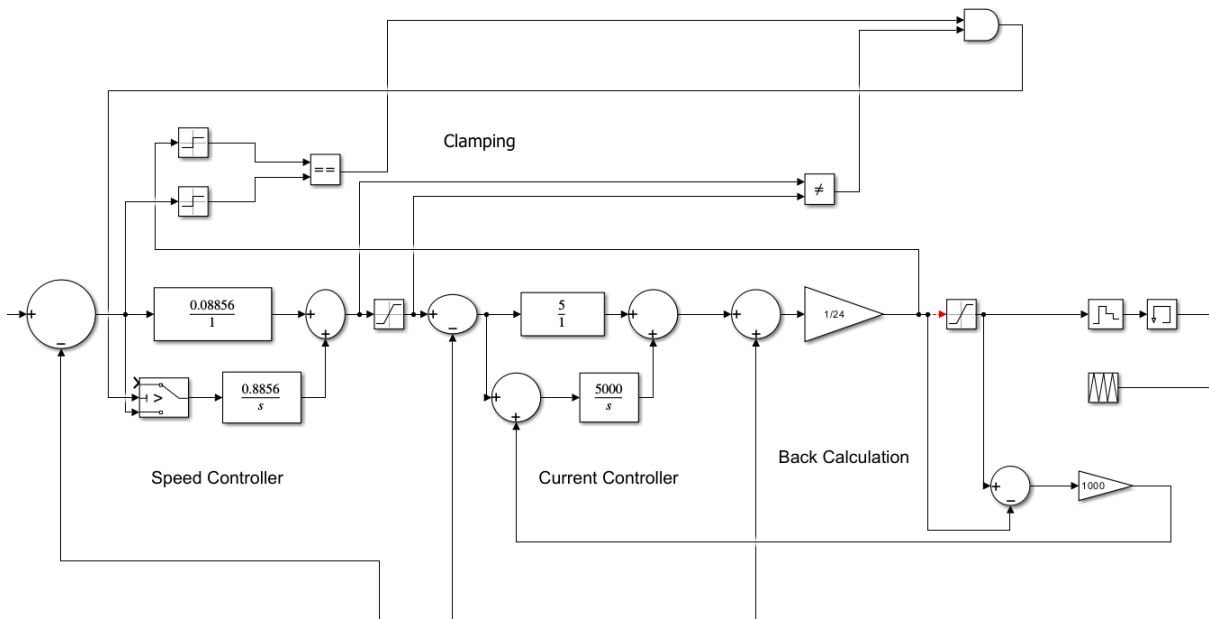


Figure 4.11: Current Controller Anti-Windup Implementation

Whenever duty-ratio saturation occurs, the back-calculation path automatically reduces the accumulated integral action. Consequently, the controller recovers more quickly from saturation and excessive overshoot is avoided.

Thus, the final cascaded control structure contains two separate protection mechanisms:

1. Current limiting at the output of the speed controller to protect the motor and converter from excessive current demand.
2. Back-calculation anti-windup in the current controller to prevent integral accumulation caused by duty-ratio saturation.

Together, these modifications significantly improve the practical transient response of the drive system.

4.8 Feedforward Compensation

Up to this point, all control action was generated through feedback.

In a feedback system, the controller first waits for an error to occur before generating corrective action. Although feedback provides excellent disturbance rejection and robustness, it inherently reacts only after the error has already appeared.

To further improve the transient response, Feedforward Compensation was introduced.

The idea behind feedforward control is simple. Instead of waiting for an error to develop, a portion of the required control effort is estimated directly from the motor model and applied in advance.

Recall the armature voltage equation

$$V_a = R_a I_a + L_a \frac{dI_a}{dt} + E_b$$

where

$$E_b = K_e \omega$$

The back EMF term represents the voltage required simply to maintain the existing motor speed.

Therefore, rather than forcing the feedback controller to generate the entire control effort, the estimated back EMF component can be supplied directly through a feedforward path.

The feedback controller then only needs to compensate for modelling inaccuracies, disturbances and parameter variations.

This significantly reduces the burden on the controller and improves the transient response.

The feedforward implementation used in this project is shown below.

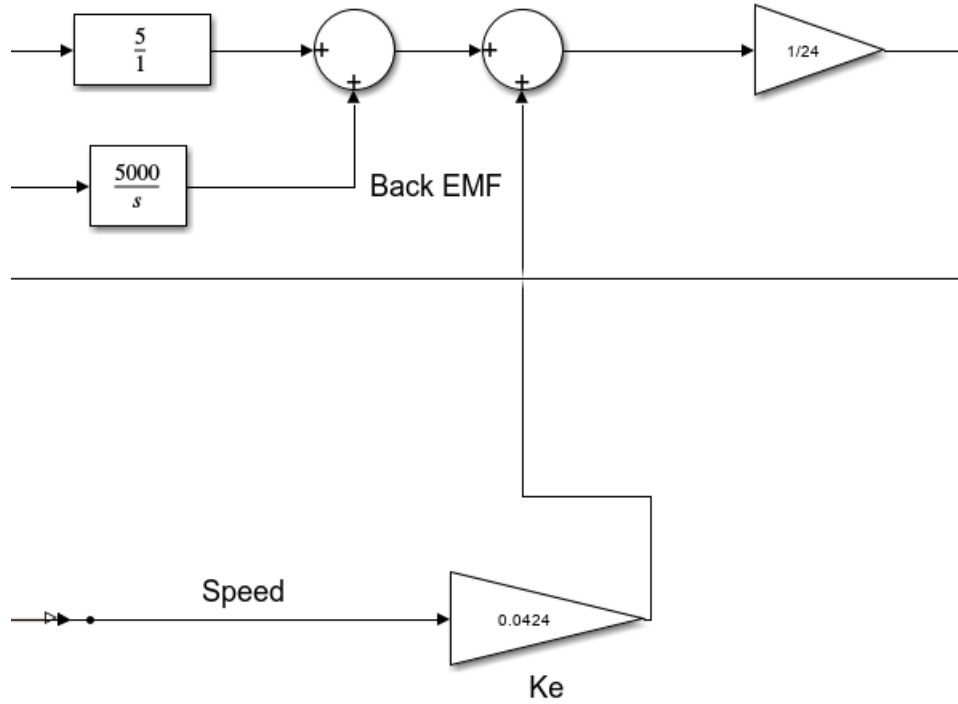


Figure 4.12: Feedforward Compensation Structure

After introducing feedforward compensation, the controller required less corrective action during speed transitions and disturbance conditions, resulting in improved dynamic performance.

4.9 Current Measurement Filtering

After implementing the cascaded speed-current control structure, the output of the outer speed controller becomes the reference current I_{ref} for the inner current loop. The current controller therefore regulates the error

$$e_i = I_{ref} - I_a$$

where I_a is the measured armature current.

In a PWM-based motor drive, the armature current is not perfectly smooth. Due to switching action of the power converter, the measured current contains high-frequency ripple components superimposed on the average current value. Although these ripple components are a normal consequence of PWM operation, they can adversely affect the controller if directly fed back into the current control loop.

Without filtering, the current controller receives a rapidly varying error signal and attempts to respond to high-frequency switching ripple that does not represent actual changes in motor torque demand. This increases controller activity, introduces unnecessary control effort, and may degrade overall system performance.

The designed current-loop crossover frequency was approximately

$$\omega_{c,current} = 1170 \text{ rad/s}$$

which corresponds to

$$f_{c,current} = \frac{1170}{2\pi}$$

$$f_{c,current} \approx 186 \text{ Hz}$$

The PWM switching frequency was

$$f_{sw} = 2000 \text{ Hz}$$

Since the switching ripple occurs at frequencies significantly higher than the current-loop bandwidth, the controller should respond only to the average current while ignoring the high-frequency ripple components.

To achieve this, a first-order low-pass filter was introduced in the current feedback path.

$$G_f(s) = \frac{1}{s\tau + 1}$$

The filter cutoff frequency was selected sufficiently higher than the current-loop crossover frequency to avoid affecting the desired control dynamics while still attenuating switching ripple. A practical guideline is to choose the filter cutoff frequency several times larger than the control bandwidth.

Therefore,

$$f_f \approx 5 f_{c,current}$$

$$f_f \approx 5 \times 186$$

$$f_f \approx 1000 \text{ Hz}$$

The filter time constant is given by

$$\tau = \frac{1}{2\pi f_f}$$

Substituting $f_f = 1000 \text{ Hz}$,

$$\tau = \frac{1}{2\pi \times 1000}$$

$$\tau \approx 0.00016s$$

Thus, the implemented filter transfer function became

$$G_f(s) = \frac{1}{0.00016s + 1}$$

The filter significantly reduced switching ripple in the measured current signal while preserving the desired current-control dynamics.

As a result, the current controller received a cleaner feedback signal and produced smoother control action.

4.10 Carrier Wave Modification

Initially, PWM generation was implemented using a sawtooth carrier waveform.

A sawtooth carrier is simple to implement and provides the required duty-ratio generation. However, practical motor drives commonly employ triangular carrier waveforms because they provide more symmetrical switching behaviour.

Therefore, the PWM generator was modified by replacing the sawtooth carrier with a triangular carrier.

The switching frequency and carrier amplitude were kept unchanged.

Since only the carrier shape was modified, the average converter gain remained unchanged.

Therefore,

$$\frac{V_a}{V_c} = \frac{V_g}{V_m}$$

remained identical to the previously derived relationship.

Consequently, no controller redesign or retuning was required.

Although the average motor response remains nearly unchanged, the triangular carrier provides a more realistic representation of industrial PWM implementation and produces symmetrical switching intervals.

4.11 Final Controller Performance

After introducing

- Duty-ratio based implementation
- Integrator clamping anti-windup
- Cascaded speed-current control
- Current limiting
- Back-calculation anti-windup
- Feedforward compensation
- Current measurement filtering
- Triangular carrier PWM generation

the final controller architecture was obtained.

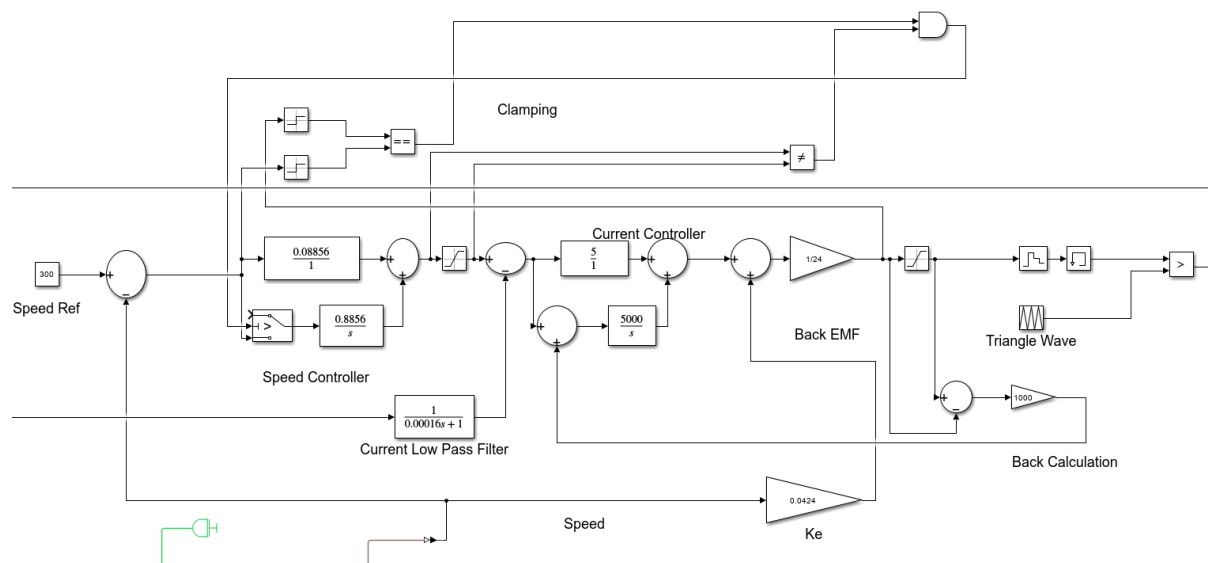


Figure 4.15: Final Controller Architecture

The resulting speed response is shown below.

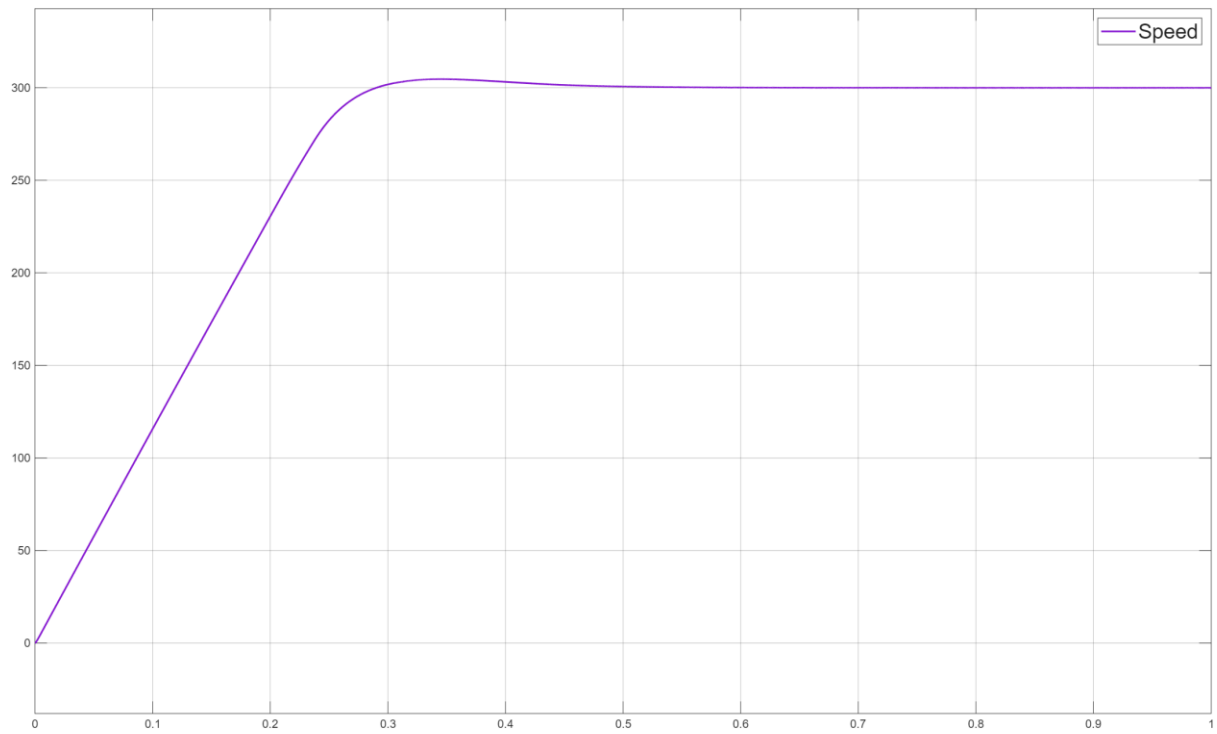


Figure 4.16: Final Closed-Loop Speed Response

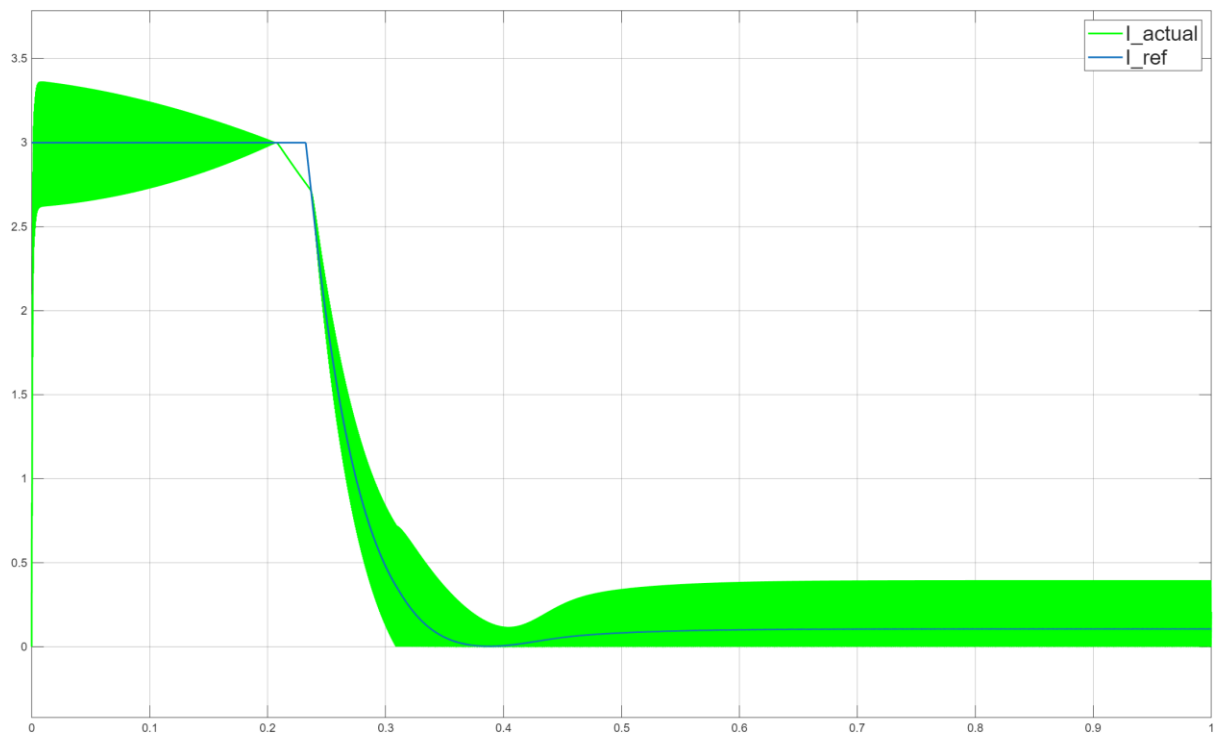


Figure 4.17: Current Reference Tracking Performance of the Final Controller

Figure 4.17 shows the current tracking performance of the inner current loop. It can be observed that the actual armature current closely follows the reference current generated by the speed controller. This confirms that the current controller

provides fast and accurate current regulation, allowing the speed controller to effectively command motor torque.

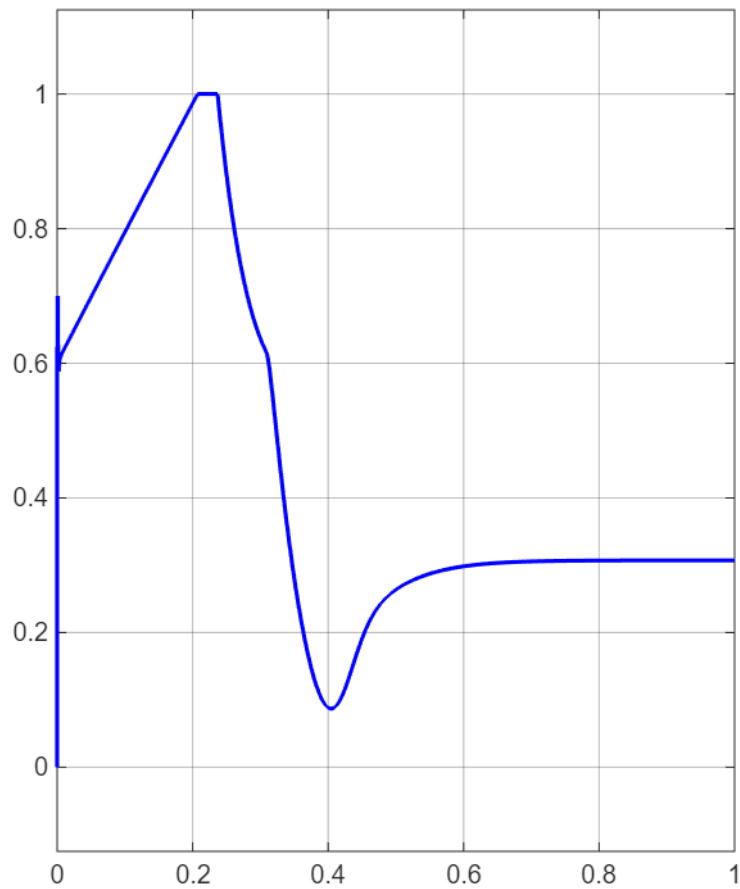


Figure 4.18: PWM Duty-Ratio Response of the Final Controller

Figure 4.18 shows the duty-ratio command generated by the controller. During startup, the duty ratio reaches its upper limit as the controller attempts to accelerate the motor rapidly. Once the speed approaches its reference value, the duty ratio decreases and settles to the value required to maintain the desired operating speed. The saturation limits ensure that the duty ratio always remains within the physically realizable range of the converter.

The corresponding performance metrics are summarized in Table 4.2.

Table 4.2: Final Controller Performance

Peak Speed	304.6646 rad/s
Overshoot	1.5549 %
Steady-State Speed	299.9953 rad/s
Steady-State Error	0.001583 %
Settling Time	0.2706 s

Comparing these results with the initial controller implementation, a substantial improvement can be observed.

The settling time was reduced from approximately 5.37 s to 0.27 s while the overshoot was reduced from approximately 19.4 % to 1.55 %.

At the same time, excellent steady-state accuracy was maintained.

The final controller therefore successfully satisfies the desired performance objectives while remaining compatible with practical converter and motor constraints.

Having obtained the final controller structure, the next chapter focuses on validating its performance under different operating conditions and comparing the various controller configurations developed throughout this project.

4.12 Comparison of Controller Configurations

Throughout the controller development process, several modifications were introduced to improve the practical performance of the drive system. Table 4.3 summarizes the performance obtained at various stages of controller development.

Table 4.3: Comparison of Controller Configurations

Performance Metric	Frequency-Domain Controller	PI Controller	Anti-Windup + Duty-Ratio Implementation	Final Cascaded Controller
Peak Speed (rad/s)	358.1783	358.2332	334.1434	304.6646
Overshoot (%)	19.3928	19.4111	11.3811	1.5549
Steady-State Speed (rad/s)	299.9821	299.9980	300.0039	299.9953
Steady-State Error (%)	0.005973	0.000674	0.001309	0.001583
Settling Time (s)	5.3661	5.3701	3.2392	0.2706

It can be observed that the introduction of integral action alone produced only marginal changes in performance. The primary improvement was obtained after introducing anti-windup compensation and duty-ratio based implementation, which significantly reduced overshoot and settling time.

The most substantial improvement was achieved after introducing the cascaded speed-current control architecture together with current limiting, back-calculation anti-windup, feedforward compensation, current filtering and triangular-carrier PWM generation.

Compared with the original frequency-domain controller, the final controller reduced the settling time from approximately 5.37 s to 0.27 s while reducing overshoot from approximately 19.4% to 1.55%. At the same time, excellent steady-state accuracy was maintained.

These results demonstrate the importance of incorporating practical implementation considerations into controller design and validate the effectiveness of the final cascaded control architecture.

5. Validation of the Proposed Control System

5.1 Introduction

After designing the cascaded speed-current control system and incorporating practical enhancements such as current limiting, anti-windup compensation, feedforward compensation, current filtering and PWM implementation, additional validation tests were performed to evaluate the dynamic performance and robustness of the controller.

The objective of these tests was not only to verify reference tracking performance but also to examine controller behaviour under realistic operating conditions such as changing speed commands and external load disturbances.

Two primary validation scenarios were considered:

1. Speed Reference Change Test
2. Load Torque Disturbance Rejection Test

These tests provide insight into the controller's ability to maintain the desired speed, reject disturbances and generate the required electromagnetic torque under varying operating conditions.

In addition, a negative load torque test was performed to investigate the limitations of the implemented one-quadrant converter and motivate the need for a four-quadrant drive topology.

5.2 Speed Reference Tracking and Positive Load Disturbance Test

In practical motor drive applications, the desired operating speed is rarely constant. Motors are frequently required to accelerate, decelerate and establish new operating points while maintaining stability and acceptable transient performance. Furthermore, the mechanical load acting on the motor may change during operation, requiring the controller to continuously adjust the motor torque in order to maintain the desired speed.

To evaluate both the tracking capability and disturbance rejection performance of the proposed cascaded controller, the speed reference and load torque were intentionally varied during operation.

Initially, the motor was operated at

$$50 \text{ rad/s}$$

At

$$t = 2 \text{ s}$$

the reference speed was increased to

$$400 \text{ rad/s}$$

The controller was therefore required to accelerate the motor and establish the new operating speed while maintaining stability and minimizing transient error.

To further evaluate disturbance rejection capability, a positive load torque of

$$0.02 \text{ N} \cdot \text{m}$$

was applied at

$$t = 6 \text{ s}$$

while maintaining the speed reference constant.

The primary objectives of this test were:

- To verify accurate speed tracking.
- To evaluate transient performance during speed transitions.
- To examine controller behaviour under load disturbances.
- To confirm proper coordination between the outer speed loop and inner current loop.

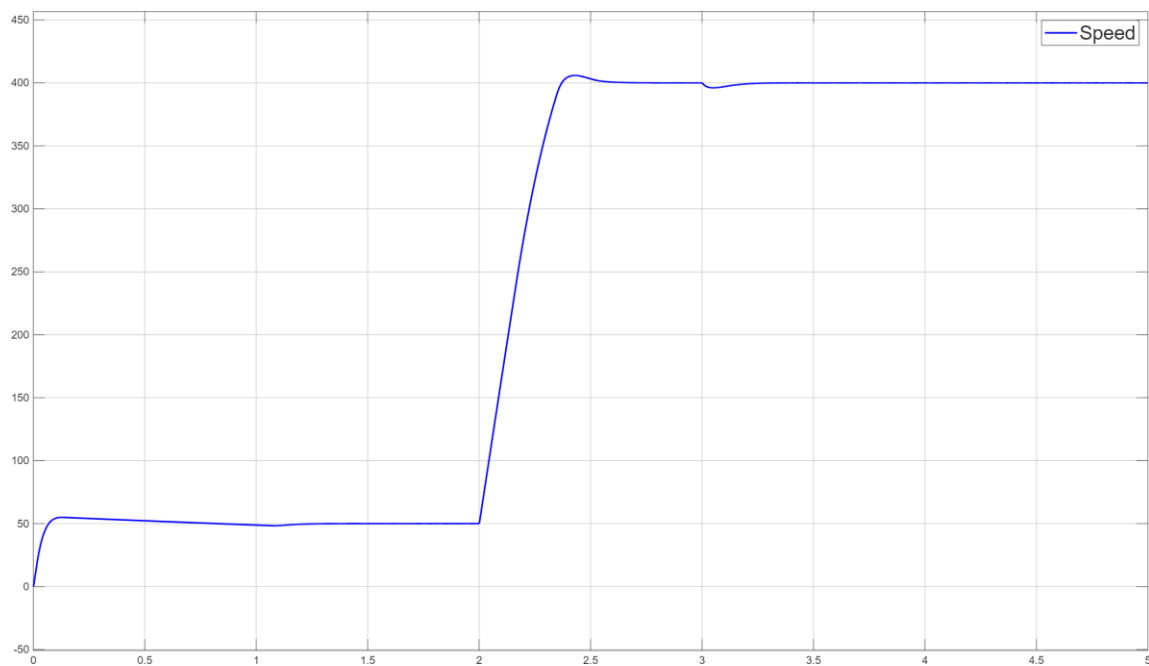


Figure 5.1: Speed Response during Reference Change from 50 rad/s to 400 rad/s with Positive Load Torque Disturbance

As shown in Figure 5.1, the controller successfully tracks the commanded speed and rapidly accelerates the motor from 50 rad/s to 400 rad/s when the reference command is changed at ($t=2s$).

The response exhibits low overshoot, fast settling and negligible steady-state error, demonstrating effective coordination between the speed and current control loops.

At

$$t = 6 s$$

the positive load torque disturbance is introduced. The additional load torque initially causes a small reduction in motor speed due to the increased mechanical demand.

The speed controller immediately detects the resulting speed error and generates the additional control effort required to compensate for the disturbance. Consequently, the motor speed quickly returns to the desired operating point after a brief transient period.

This behaviour demonstrates effective disturbance rejection and confirms that the proposed cascaded controller can maintain accurate speed regulation even when subjected to changing load conditions.

The ability to automatically compensate for variations in load torque is particularly important in practical applications such as conveyor systems, elevators, rolling mills and industrial drive systems where mechanical loading conditions may vary continuously during operation.

5.3 Deceleration Test and Limitation of One-Quadrant Operation

Although the previous test demonstrated excellent acceleration and disturbance rejection performance, an additional validation study was performed to investigate controller behaviour under conditions requiring braking action.

In the first test, the motor was initially operated at

$$400 \text{ rad/s}$$

and at

$$t = 2 s$$

the speed reference was reduced to

$$50 \text{ rad/s}$$

while maintaining the load torque unchanged.

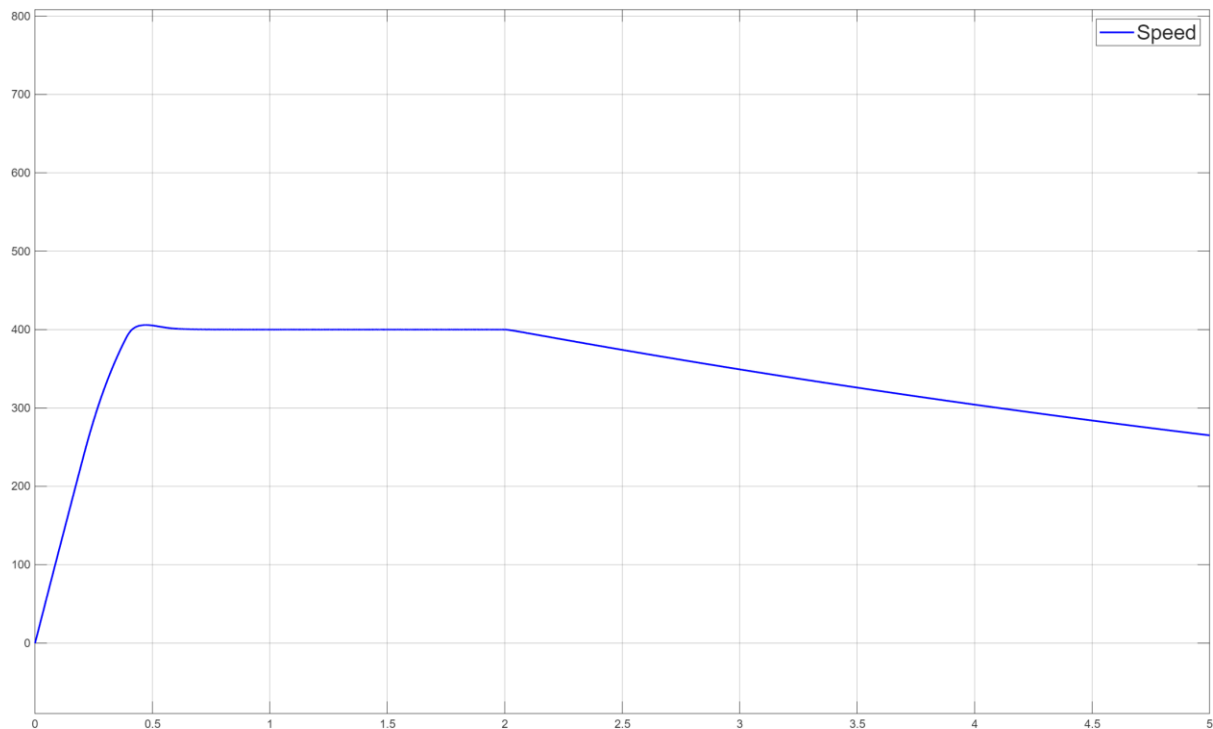


Figure 5.2: Speed Response during Reference Change from 400 rad/s to 50 rad/s

As shown in Figure 5.2, the motor speed begins decreasing after the reference is changed from 400 rad/s to 50 rad/s. However, the reduction in speed is considerably slower than the acceleration response observed in the previous test.

This behaviour occurs because the implemented converter is a one-quadrant drive and therefore cannot generate negative armature current or negative electromagnetic torque. When the speed reference is reduced, the controller attempts to decelerate the motor. However, active braking is not available and the motor can only slow down through its natural mechanical losses and the applied load torque.

Consequently, the speed decreases gradually rather than rapidly approaching the new reference value. Within the simulation interval, the motor does not fully reach the commanded speed of 50 rad/s. Nevertheless, the continuously decreasing speed indicates that the motor is attempting to settle towards the new operating point.

This result highlights an important limitation of one-quadrant operation. While acceleration can be achieved rapidly through positive motor torque, deceleration is significantly slower because negative torque cannot be generated.

A second test was then performed to further investigate this limitation.

The speed reference was maintained constant at

$$400 \text{ rad/s}$$

while the load torque was changed at

$$t = 2 \text{ s}$$

from

$$0 \text{ to } -0.02 \text{ N} \cdot \text{m}$$

A negative load torque assists motor rotation rather than opposing it. Consequently, the load itself contributes mechanical power to the motor shaft.

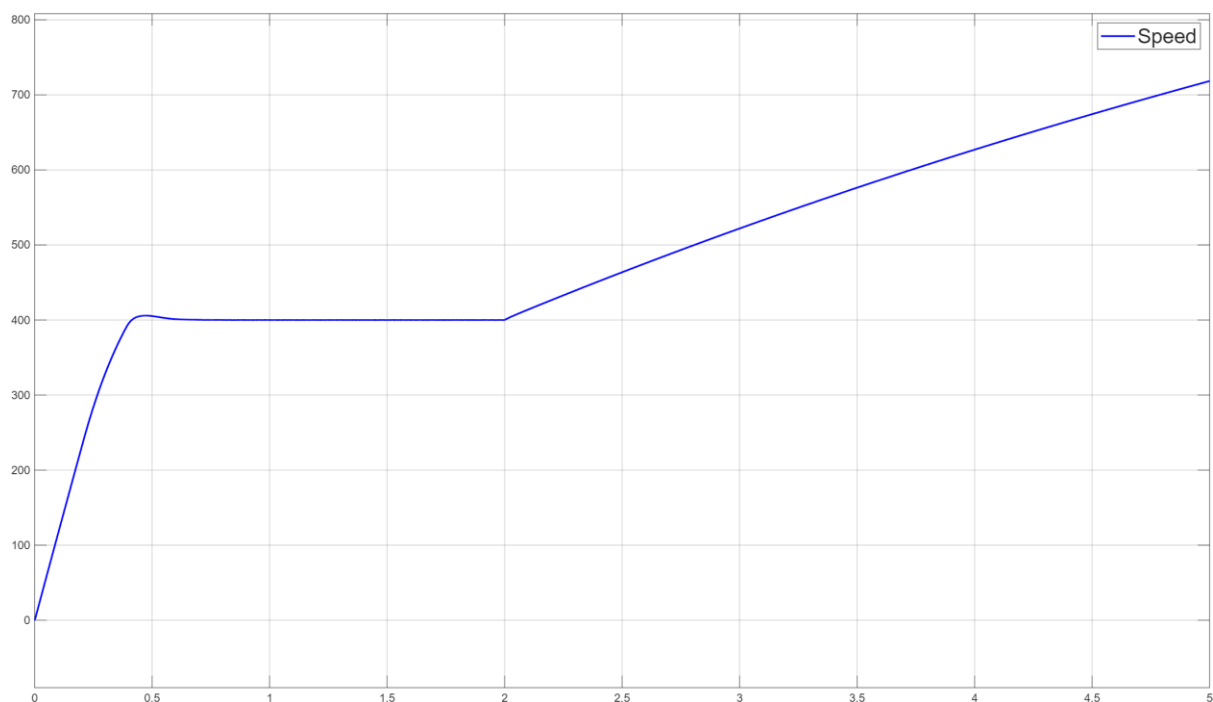


Figure 5.3: Speed Response under Negative Load Torque Disturbance

As shown in Figure 5.3, the motor speed continuously increases after the negative load torque is applied. The assisting load torque accelerates the motor and causes the speed to rise above its reference value.

Under these conditions, the controller effectively requires negative electromagnetic torque in order to oppose the assisting load and maintain the commanded speed. However, since the implemented converter can only operate in the first quadrant, negative armature current and negative motor torque cannot be produced.

As a result, the controller is unable to actively brake the motor and the speed continues to increase under the influence of the assisting load torque.

This behaviour does not indicate a controller failure. Rather, it highlights a fundamental limitation of one-quadrant converter operation.

To provide active braking, regenerative braking and bidirectional torque capability, a four-quadrant drive topology is required. A four-quadrant converter would allow both positive and negative armature voltage as well as positive and negative armature current. Consequently, the motor could generate negative electromagnetic torque whenever required, enabling rapid deceleration and improved speed regulation under all operating conditions.

Therefore, the observed behaviour provides strong motivation for extending the present work toward the implementation of a four-quadrant DC motor drive, which is presented in the next chapter.

5.4 Motivation for Four-Quadrant Drive

The validation tests presented in this chapter demonstrate that the proposed cascaded speed-current controller provides excellent performance during normal operating conditions. The controller successfully tracks speed commands, rejects positive load disturbances and maintains negligible steady-state error.

However, the deceleration and assisting-load tests reveal an important limitation of the implemented converter topology.

When the motor is required to accelerate, positive electromagnetic torque can be generated and the desired speed is achieved rapidly. Similarly, when additional opposing load torque is applied, the controller can increase armature current and generate the additional torque required to maintain speed regulation.

In contrast, when the speed reference is reduced or when an assisting load torque is applied, the controller effectively requires negative electromagnetic torque in order to decelerate the motor. Since the implemented converter is a one-quadrant drive, only positive armature voltage and positive armature current can be produced. Consequently, negative motor torque cannot be generated.

As a result, motor deceleration is limited to the natural mechanical losses of the system, and active braking is not possible. Under assisting-load conditions, the motor speed may continue to increase even though the controller attempts to reduce it.

These observations highlight that the limitation is not associated with the controller design itself. Rather, it arises from the power converter topology being restricted to first-quadrant operation.

To overcome this limitation, a converter capable of producing both positive and negative motor torque is required. Such a system is known as a Four-Quadrant DC Motor Drive.

A four-quadrant drive permits operation in all regions of the torque-speed plane and provides:

- Forward motoring operation
- Forward regenerative braking operation
- Reverse motoring operation
- Reverse regenerative braking operation

Consequently, the drive can accelerate, decelerate, reverse direction and perform regenerative braking while maintaining precise speed regulation.

Therefore, the next stage of this project focuses on extending the present one-quadrant drive into a four-quadrant DC motor drive capable of bidirectional torque and speed control.

The design, modelling and implementation of the four-quadrant drive are presented in the next chapter.

6. Four-Quadrant Drive Implementation

6.1 Introduction and Converter Modifications

The validation tests presented in Chapter 5 highlighted an important limitation of the one-quadrant converter. Although the controller was capable of accurately tracking speed references and rejecting positive load disturbances, it was unable to generate negative electromagnetic torque.

Consequently, active braking was not possible and motor deceleration depended entirely on the natural mechanical losses of the system. Furthermore, under assisting-load conditions, the motor speed continued to increase because the converter could not produce the negative torque required to oppose the external load.

To overcome this limitation, the power stage was replaced with a Four-Quadrant H-Bridge Converter.

The objective was to enable both bidirectional armature voltage and bidirectional armature current while retaining the cascaded speed-current control architecture developed previously.

Since the speed controller, current controller, anti-windup mechanisms, feedforward compensation and current filtering had already demonstrated satisfactory performance, the overall control structure was preserved. Only the converter interface and operating limits were modified to enable operation in all four quadrants of the torque-speed plane.

In the one-quadrant implementation, the current reference was restricted to positive values only

$$0 \leq I_{ref} \leq 3A$$

This restriction was necessary because the converter could only supply positive armature current. Consequently, braking torque could not be produced.

For four-quadrant operation, the current reference limit was modified to

$$-3A \leq I_{ref} \leq 3A$$

thereby allowing both positive and negative current.

Positive current produces positive electromagnetic torque and therefore motoring operation, while negative current produces negative electromagnetic torque and therefore braking operation.

Similarly, the duty-cycle limitation was changed from

$$0 \leq d \leq 1$$

to

$$-1 \leq d \leq 1$$

which enables both positive and negative average armature voltage.

The controller output duty cycle was then separated into two components:

$$d \rightarrow |d| + \text{sign}(d)$$

The magnitude component

$$|d|$$

was used to generate the PWM duty ratio, while the sign component determined the direction of power flow through the H-bridge converter.

This approach allows the previously developed PWM generation method to be retained without modification. Since only the magnitude of the duty cycle is compared with the carrier waveform, the effective PWM gain remains unchanged from the one-quadrant implementation.

Consequently, the previously designed speed controller and current controller could be retained without further redesign.

The modifications introduced for four-quadrant operation are summarized in Table 6.1.

Table 6.1: Comparison between One-Quadrant and Four-Quadrant Operation

Parameter	One-Quadrant Drive	Four-Quadrant Drive
Current Reference Limit	0 to 3 A	-3 to 3 A
Duty Ratio Limit	0 to 1	-1 to 1
Armature Current	Positive Only	Positive and Negative
Electromagnetic Torque	Positive Only	Positive and Negative
Active Braking	Not Available	Available
Regenerative Braking	Not Available	Available

By introducing bidirectional current and bidirectional voltage capability, the drive becomes capable of both motoring and braking operation while preserving the controller structure developed in the previous chapters.

6.2 H-Bridge Switching Strategy

The H-bridge converter consists of four controlled switches denoted by S_1, S_2, S_3, S_4

By selecting appropriate switch pairs, either positive or negative voltage can be applied across the motor armature terminals.

The sign of the controller duty cycle determines the direction of operation.

For positive duty-cycle values

$$d > 0$$

the converter operates in the forward direction. In this mode, the switch pair $S_1 - S_4$ is activated and positive average voltage is applied across the armature.

For negative duty-cycle values

$$d < 0$$

the converter operates in the reverse direction. In this mode, the switch pair $S_2 - S_3$ is activated and negative average voltage is applied across the armature.

The magnitude of the duty cycle

$$|d|$$

is used for PWM generation, while the sign of the duty cycle determines which switch pair is active.

The switching logic used in the implementation is summarized in Table 6.2.

Table 6.2: H-Bridge Switching Logic

Duty Cycle Sign	Active Switch Pair	Mode of Operation
(d>0)	$S_1 - S_4$	Forward Operation
(d<0)	$S_2 - S_3$	Reverse Operation

The use of sign-based direction selection and magnitude-based PWM generation significantly simplifies the control implementation while maintaining compatibility with the previously developed cascaded control architecture.

Special care must be taken in H-bridge converters to prevent simultaneous conduction of switches belonging to the same converter leg.

Such a condition, commonly known as Shoot-Through, creates a direct short circuit across the DC supply and may result in excessive current and damage to the power semiconductor devices.

To avoid this issue, complementary gate signals were generated using dedicated switching logic. In addition, a dead-time interval was incorporated into the switching sequence.

The dead-time was implemented using a custom MATLAB Function block. Whenever a switch was commanded to turn ON, the corresponding gate signal was delayed by a predefined interval. However, switch turn-OFF was performed immediately. The MATLAB Function code used to implement the dead-time generation logic for the H-bridge switching scheme is provided in **Appendix E**.

This strategy ensures that one switch is completely turned OFF before its complementary switch is allowed to turn ON, thereby eliminating the possibility of shoot-through.

The resulting switching architecture is shown below.

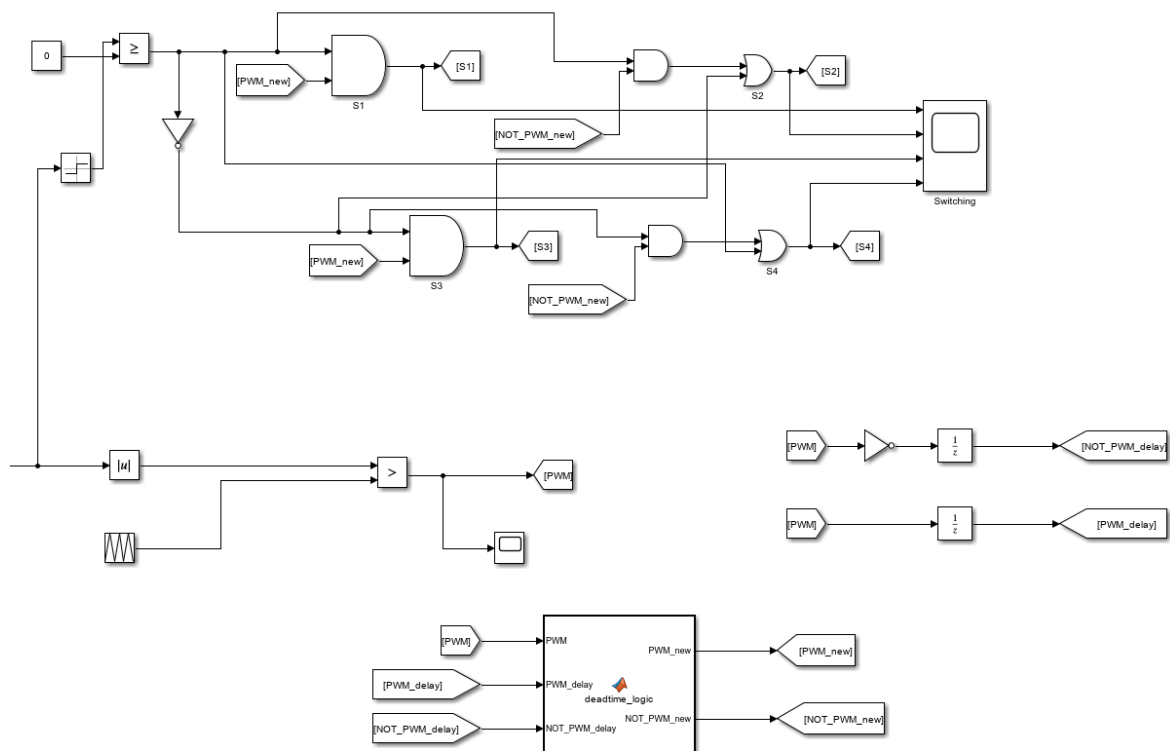


Figure 6.2: Final H-Bridge Switching Logic with Dead-Time Implementation

The successful implementation of the H-bridge switching logic enables safe bidirectional voltage application and forms the foundation for four-quadrant motor operation.

6.3 Four-Quadrant Validation Test

To verify the effectiveness of the proposed four-quadrant implementation, a validation test was performed under operating conditions that could not be handled satisfactorily by the one-quadrant drive.

Initially, the motor was operated at

$$400 \text{ rad/s}$$

At

$$t = 3s$$

the speed reference was reduced to

$$50 \text{ rad/s}$$

requiring the drive to actively decelerate the motor.

To further evaluate operation under assisting-load conditions, the load torque was changed at

$$t = 6s$$

from

$$0 \text{ to } -0.02 \text{ N} \cdot \text{m}$$

The negative load torque assists motor rotation and therefore tends to increase the motor speed.

The primary objectives of this test were:

- To verify active braking capability.
- To evaluate bidirectional current control.
- To examine controller behaviour under assisting-load conditions.
- To validate the operation of the H-bridge switching strategy.

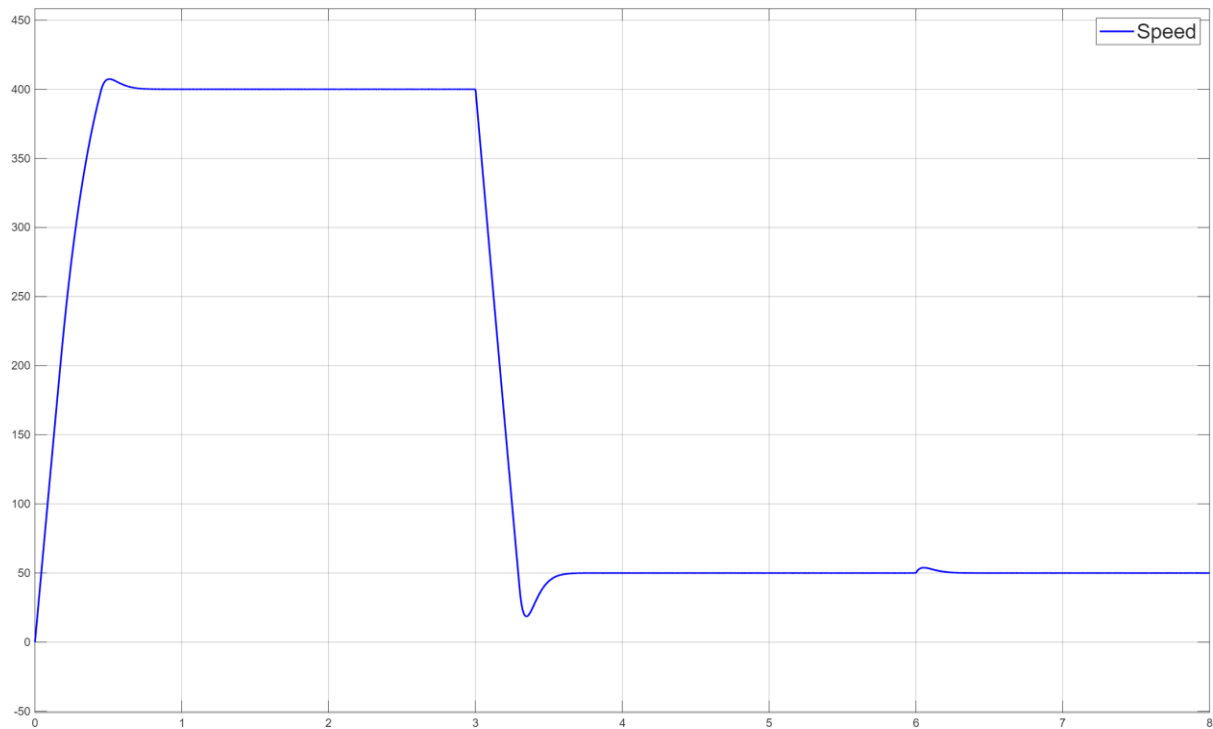


Figure 6.3: Speed Response of the Four-Quadrant Drive

Unlike the one-quadrant implementation presented in Chapter 5, the motor now decelerates rapidly towards the new operating point after the speed reference is reduced.

This improvement is possible because the H-bridge converter allows negative armature current and therefore negative electromagnetic torque to be generated whenever braking action is required.

As a result, the controller is no longer limited to natural mechanical deceleration and can actively regulate motor speed during speed reductions.

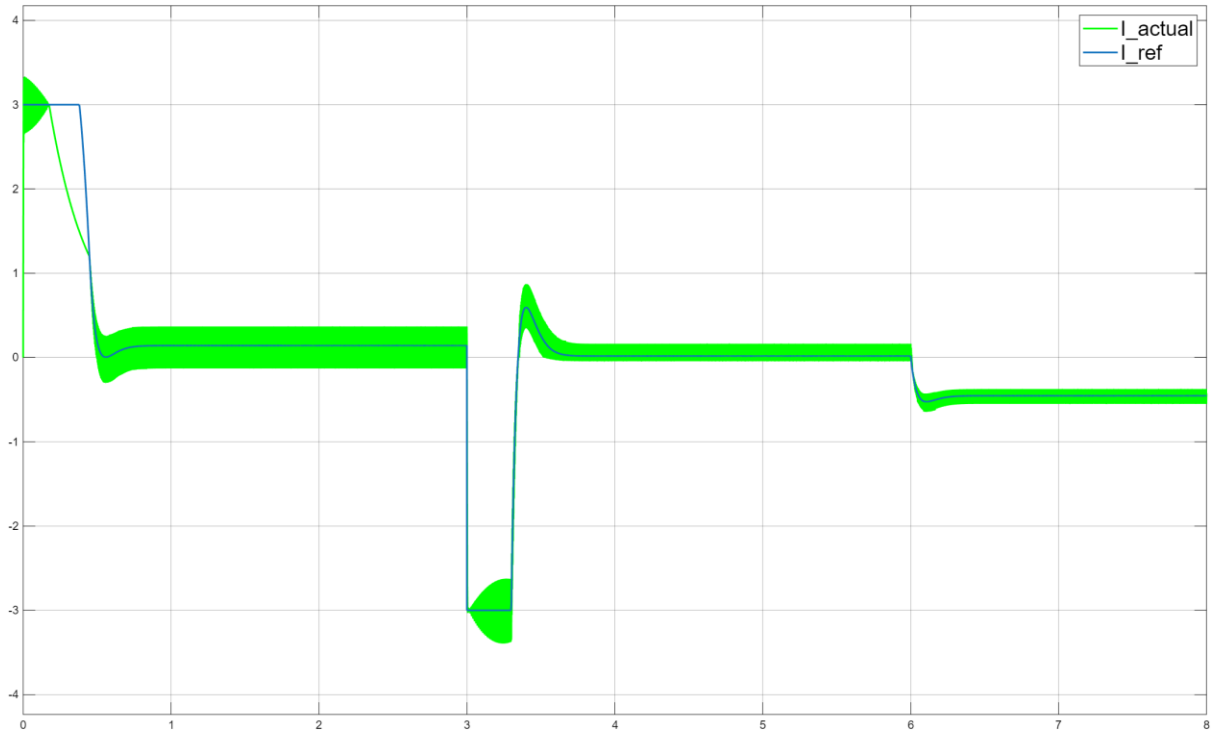


Figure 6.4: Current Reference Tracking Performance

Figure 6.4 shows the performance of the inner current loop during both motoring and braking operation.

The armature current closely follows the commanded current reference, demonstrating effective current regulation and confirming the proper operation of the cascaded control architecture.

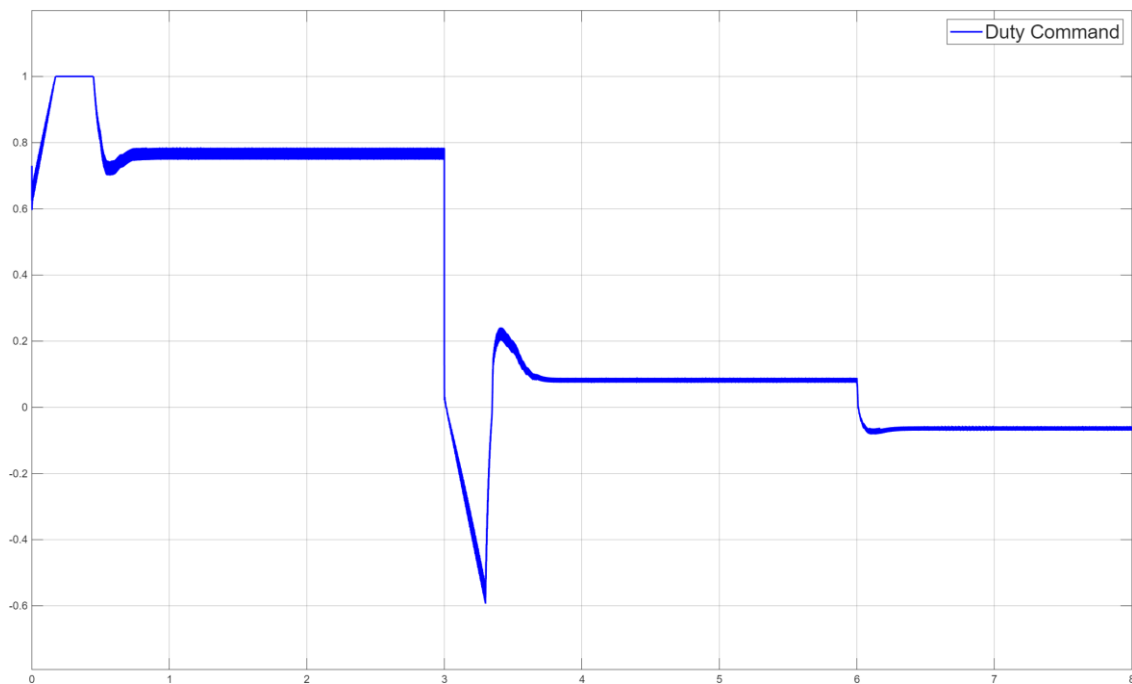


Figure 6.5: Duty Command Response during Four-Quadrant Operation

The duty command becomes negative whenever braking torque is required.

Positive duty values correspond to forward motoring operation, while negative duty values correspond to reverse voltage application through the H-bridge converter.

This demonstrates successful bidirectional control of the converter output voltage.

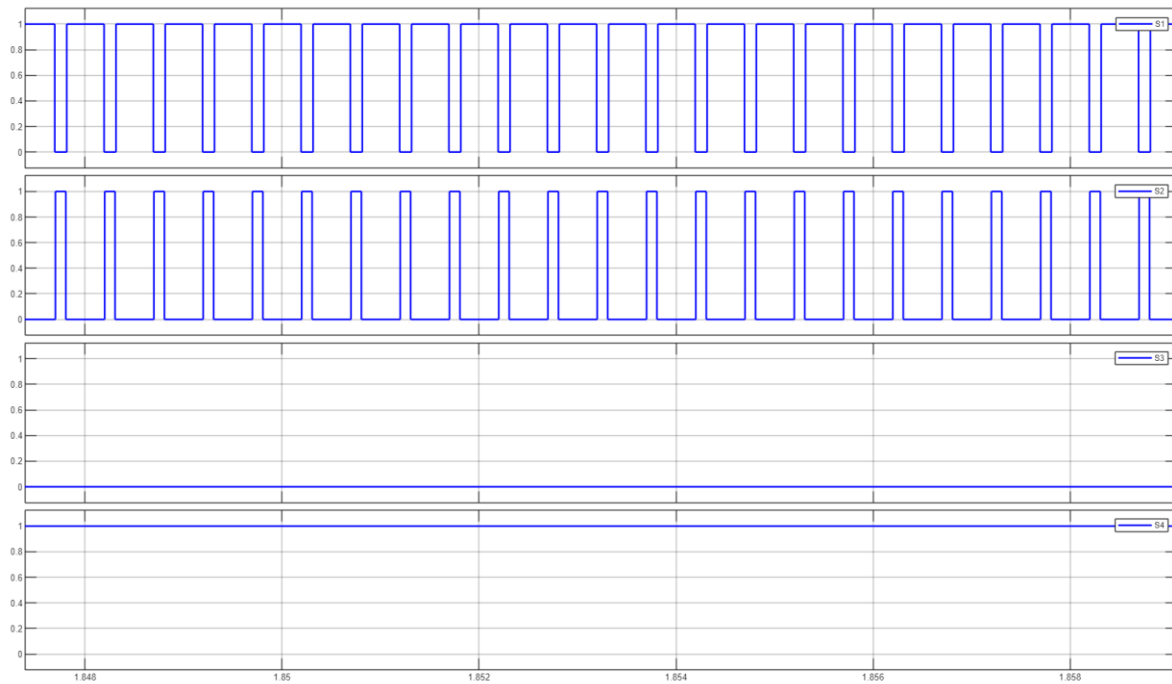


Figure 6.6: H-Bridge Switching Signals

Figure 6.6 shows the switching signals generated for the four power semiconductor devices.

The gate signals correctly change according to the sign of the duty command, confirming the proper operation of the switching logic and dead-time implementation.

The results demonstrate that the proposed four-quadrant drive successfully overcomes the braking limitations observed in the one-quadrant implementation and provides stable operation under both deceleration and assisting-load conditions.

6.4 Regenerative Braking and Bidirectional Operation

One of the most important advantages of a four-quadrant drive is the ability to perform regenerative braking and reverse motoring operation.

To evaluate this capability, a large speed reversal test was performed.

The motor was initially operating at

$$+400 \text{ rad/s}$$

At

$$t = 3 \text{ s}$$

the speed reference was changed to

$$-400 \text{ rad/s}$$

This test requires the drive to decelerate the motor, reverse the direction of rotation and subsequently accelerate the motor in the opposite direction.



Figure 6.7: Speed Response during Speed Reversal from +400 rad/s to -400 rad/s

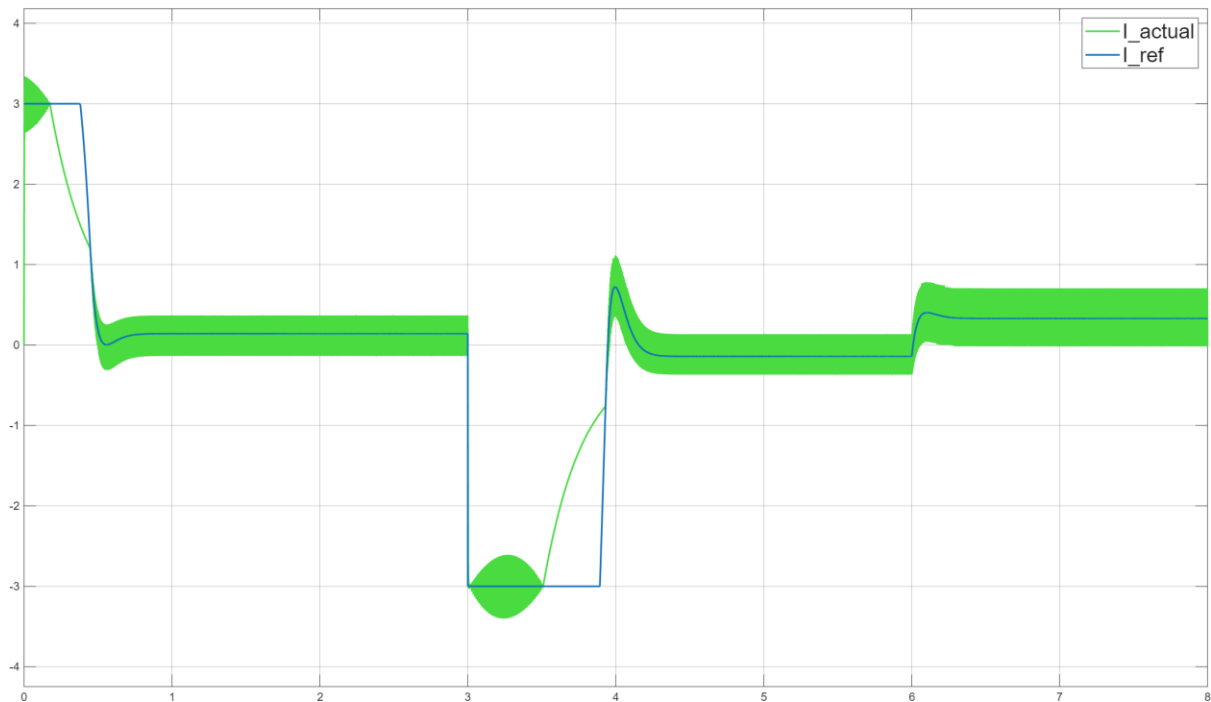


Figure 6.8: Armature Current Response during Regenerative Braking and Reverse Motoring

When the reference speed is changed, the speed controller immediately demands negative current.

Consequently, the armature current reaches the negative current limit and the motor begins generating negative electromagnetic torque.

During the interval in which the motor speed remains positive while the armature current is negative, the drive operates in the **Forward Regenerative Braking Region (Quadrant II)**.

In this operating mode, the motor functions as a generator and electrical energy is returned to the source while the motor is actively decelerated.

As the motor speed approaches zero, the regenerative braking process continues until the direction of rotation reverses.

After crossing zero speed, the drive enters the **Reverse Motoring Region (Quadrant III)**.

At this stage, the motor accelerates in the reverse direction and eventually settles at the commanded speed of

$$-400 \text{ rad/s}$$

The results confirm successful operation in multiple quadrants of the torque-speed plane and demonstrate the ability of the proposed drive to perform regenerative braking, active deceleration and reverse motoring operation.

This capability represents a significant improvement over the one-quadrant drive and enables precise speed control under a much wider range of operating conditions.

7. Conclusion

In this project, a complete DC motor speed control system was designed, analysed and implemented using both one-quadrant and four-quadrant converter topologies.

Initially, a mathematical model of the separately excited DC motor was developed using the electrical and mechanical equations of the machine. The resulting transfer function was used to analyse the system dynamics and design suitable controllers.

A frequency-domain controller was first designed using Bode plot based loop-shaping techniques. Although the controller satisfied the desired stability margins and steady-state accuracy requirements, practical implementation revealed significant performance degradation due to actuator saturation and converter limitations.

To overcome these issues, several practical improvements were incorporated into the control system. These included anti-windup compensation, duty-ratio based implementation, cascaded speed-current control, current limiting, feedforward compensation, current measurement filtering and improved PWM generation techniques.

The resulting cascaded control structure achieved excellent dynamic performance with low overshoot, negligible steady-state error and significantly reduced settling time. Validation tests demonstrated accurate speed tracking and effective disturbance rejection under varying operating conditions.

Further investigation revealed the inherent limitations of one-quadrant converter operation. While the controller successfully regulated motor speed during acceleration and positive load disturbances, active braking could not be achieved because negative armature current and negative electromagnetic torque were unavailable.

To overcome this limitation, the converter was upgraded to a four-quadrant H-bridge topology. Bidirectional current and voltage capability were introduced while retaining the previously developed cascaded control architecture. The modified system successfully achieved active braking, regenerative operation and reverse motoring capability.

Simulation results confirmed successful operation in multiple quadrants of the torque-speed plane. The drive demonstrated rapid deceleration, regenerative braking and bidirectional speed control, thereby overcoming the limitations observed in the one-quadrant implementation.

Overall, the project demonstrates the complete design journey of a practical DC motor drive system, beginning from mathematical modelling and controller design and progressing through implementation challenges, performance improvements and advanced four-quadrant operation. The final system provides robust speed regulation, improved dynamic performance and enhanced operational flexibility suitable for modern motor drive applications.

Key Contributions of the Project

- Developed mathematical model of separately excited DC motor.
- Designed controller using frequency-domain loop shaping.
- Investigated saturation effects and anti-windup methods.
- Implemented cascaded speed-current control.
- Added current limiting and feedforward compensation.
- Extended system from one-quadrant to four-quadrant operation.
- Demonstrated regenerative braking and reverse motoring.

Future Scope

The present work may be extended in several directions:

- Hardware implementation of the complete four-quadrant drive.
- Experimental validation using a real DC motor and power converter.
- Digital controller implementation using a microcontroller or DSP platform.
- Advanced control techniques such as State-Space Control, Model Predictive Control (MPC) and Adaptive Control.
- Regenerative energy recovery and battery-integrated drive systems.
- Extension of the proposed control architecture to BLDC and AC motor drives.

References

- [1] A. M. Bazzi, "Schematic Diagram of a Separately Excited DC Motor," ResearchGate. Available: https://www.researchgate.net/figure/Schematic-Diagram-separately-excited-DC-motor_fig1_302446129
- [2] Electrical Workbook, "First Quadrant Chopper (Class A Chopper)." Available: <https://electricalworkbook.com/first-quadrant-chopper-type-a-class-a/>
- [3] MaxEmbedded, "PWM Modulation Image." Available: <https://maxembedded.wordpress.com/wp-content/uploads/2011/08/pwmmodulation.jpg>
- [4] Electronics Stack Exchange, "Cascaded Current Control Discussion." Available: <https://electronics.stackexchange.com/questions/390725/cascaded-current-control>

APPENDICES

Appendix A

DC Motor Transfer Function Derivation

The block diagram representation of the DC motor used in this project is shown in Figure A.1.

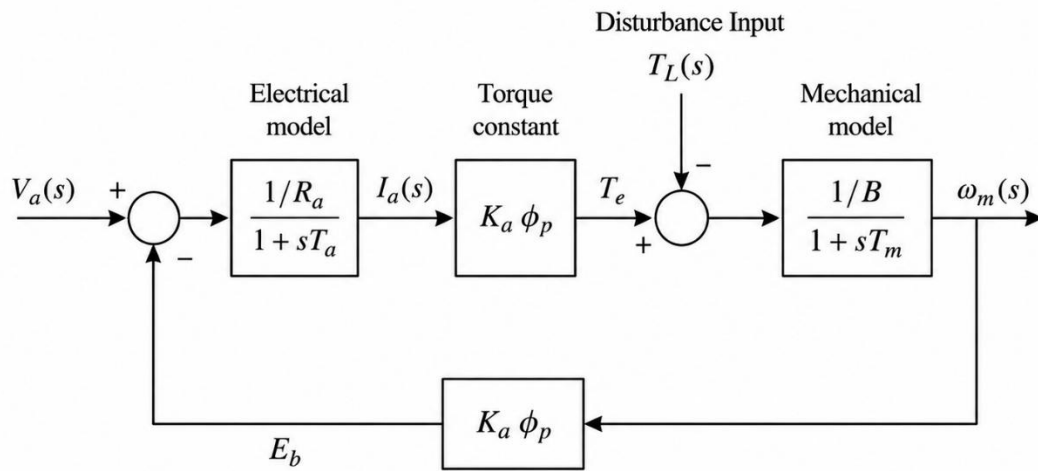


Figure A.1: Block Diagram Representation of the DC Motor

From the block diagram, the motor speed is influenced by two independent inputs:

1. Armature Voltage ($V_a(s)$)
2. Load Torque Disturbance ($T_L(s)$)

Using the principle of superposition, the motor speed can be expressed as

$$\omega_m(s) = \frac{G_1(s)}{1 + G_1(s)H_1(s)} V_a(s) + \frac{G_2(s)}{1 + G_2(s)H_2(s)} T_L(s)$$

The first term represents the contribution of the armature voltage input while the second term represents the effect of load torque disturbances.

A change in load torque produces a deviation in motor speed. This deviation appears as an error at the controller input and is compensated by the controller through appropriate adjustment of the armature voltage.

Therefore, for derivation of the nominal plant transfer function used in controller design, the disturbance input is neglected.

Hence

$$\omega_m(s) = \frac{G_1(s)}{1 + G_1(s)H_1(s)} V_a(s)$$

and therefore

$$\frac{\omega_m(s)}{V_a(s)} = \frac{G_1(s)}{1 + G_1(s)H_1(s)}$$

From Figure A.1,

$$G_1(s) = \frac{\frac{K_a \phi}{BR_a}}{(T_m s + 1)(T_a s + 1)}$$

and

$$H_1(s) = K_a \phi$$

Substituting into the closed-loop transfer function,

$$\frac{\omega_m(s)}{V_a(s)} = \frac{\frac{\frac{K_a \phi}{BR_a}}{(T_m s + 1)(T_a s + 1)}}{1 + \left(\frac{\frac{K_a \phi}{BR_a}}{(T_m s + 1)(T_a s + 1)} \right) (K_a \phi)}$$

Multiplying numerator and denominator by

$$(T_m s + 1)(T_a s + 1)$$

gives

$$\frac{\omega_m(s)}{V_a(s)} = \frac{\frac{K_a \phi}{BR_a}}{(T_m s + 1)(T_a s + 1) + \frac{(K_a \phi)^2}{BR_a}}$$

Now multiply numerator and denominator by BR_a :

$$\frac{\omega_m(s)}{V_a(s)} = \frac{K_a \phi}{BR_a(T_m s + 1)(T_a s + 1) + (K_a \phi)^2}$$

This is the complete symbolic transfer function of the DC motor relating armature voltage to motor speed and forms the basis for the controller design presented in this report.

Appendix B

Derivation of PWM Gain Model

The PWM generator used in this project converts the controller output voltage into an equivalent duty ratio by comparing it with a triangular carrier waveform.

Let

$$V_c$$

represent the controller output voltage and

$$V_m$$

represent the peak value of the triangular carrier waveform.

The duty ratio is defined as

$$D = \frac{T_{ON}}{T}$$

where

T_{ON} is the ON time of the switch and T is the switching period.

From the PWM waveform shown in Figure B.1, similar triangles can be used to relate the controller voltage and carrier voltage.

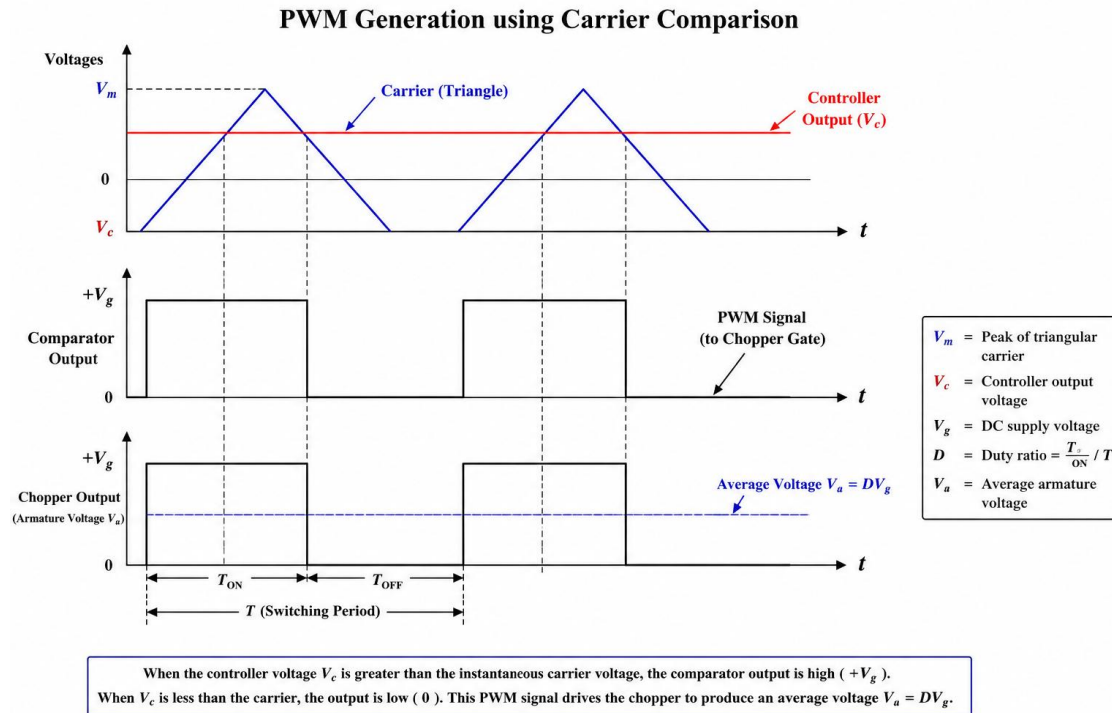


Figure B.1: PWM Generation using Carrier Comparison

Using triangle similarity,

$$\frac{V_c}{DT} = \frac{V_m}{T}$$

Multiplying both sides by (T),

$$\frac{V_c}{D} = V_m$$

Therefore,

$$D = \frac{V_c}{V_m}$$

For an ideal chopper,

$$V_a = DV_g$$

where

$$V_g$$

is the DC supply voltage and

$$V_a$$

is the average armature voltage applied to the motor.

Substituting the duty ratio expression,

$$V_a = \left(\frac{V_c}{V_m} \right) V_g$$

Rearranging,

$$\frac{V_a}{V_c} = \frac{V_g}{V_m}$$

Thus, the PWM block can be represented as a gain

$$K_{PWM} = \frac{V_a}{V_c} = \frac{V_g}{V_m}$$

Appendix C

Frequency-Domain Design Relationships

C.1 Steady-State Error

For a unity feedback system,

$$E(s) = Y^*(s) - Y(s)$$

The open-loop transfer function is

$$G_{OL}(s) = G(s)H(s)$$

The closed-loop transfer function is

$$\frac{Y(s)}{Y^*(s)} = \frac{G_{OL}(s)}{1 + G_{OL}(s)}$$

Substituting into the error equation,

$$E(s) = Y^*(s) - Y(s)$$

$$E(s) = Y^*(s) - \frac{G_{OL}(s)}{1 + G_{OL}(s)} Y^*(s)$$

$$E(s) = Y^*(s) \left(1 - \frac{G_{OL}(s)}{1 + G_{OL}(s)} \right)$$

$$E(s) = \frac{Y^*(s)}{1 + G_{OL}(s)}$$

Using the Final Value Theorem,

$$e_{ss} = \lim_{t \rightarrow \infty} e(t) = \lim_{s \rightarrow 0} s E(s)$$

For a unit step input,

$$Y^*(s) = \frac{1}{s}$$

Therefore,

$$e_{ss} = \lim_{s \rightarrow 0} \frac{1}{1 + G_{OL}(s)}$$

or

$$\boxed{e_{ss} = \lim_{s \rightarrow 0} \frac{1}{1 + G(s)H(s)}}$$

Thus increasing the low-frequency loop gain reduces the steady-state error.

C.2 Relationship Between Crossover Frequency and Settling Time

The open-loop transfer function near crossover can be approximated by

$$G_{OL}(s) = \frac{K}{\frac{s}{\omega_0} + 1}$$

For large K ,

$$\omega_c \approx K \omega_0$$

Therefore,

$$\omega_0 = \frac{\omega_c}{K}$$

Substituting into the open-loop transfer function:

$$G_{OL}(s) = \frac{K}{\frac{Ks}{\omega_c} + 1}$$

Now the closed-loop transfer function is

$$\frac{Y(s)}{Y^*(s)} = \frac{G_{OL}(s)}{1 + G_{OL}(s)}$$

Substituting:

$$\frac{Y(s)}{Y^*(s)} = \frac{\frac{K}{\frac{Ks}{\omega_c} + 1}}{1 + \frac{K}{\frac{Ks}{\omega_c} + 1}}$$

Multiplying numerator and denominator by

$$\left(\frac{Ks}{\omega_c} + 1\right)$$

gives

$$\frac{Y(s)}{Y^*(s)} = \frac{K}{\frac{Ks}{\omega_c} + 1 + K}$$

Now since

$$K \gg 1$$

then

$$1 + K \approx K$$

Hence

$$\frac{Y(s)}{Y^*(s)} = \frac{K}{K \left(\frac{s}{\omega_c} + 1 \right)}$$

which simplifies to

$$\boxed{\frac{Y(s)}{Y^*(s)} = \frac{1}{\frac{s}{\omega_c} + 1}}$$

For a unit-step input

$$Y^*(s) = \frac{1}{s}$$

therefore

$$Y(s) = \frac{1}{s} \cdot \frac{1}{\frac{s}{\omega_c} + 1}$$
$$Y(s) = \frac{\omega_c}{s(s + \omega_c)}$$

Partial fractions:

$$Y(s) = \frac{1}{s} - \frac{1}{s + \omega_c}$$

Taking inverse Laplace transform:

$$y(t) = 1 - e^{-\omega_c t}$$

Comparing with

$$y(t) = 1 - e^{-t/\tau}$$

gives

$$\boxed{\tau = \frac{1}{\omega_c}}$$

For a first-order system, the 2% settling time is approximately

$$T_s = 4\tau$$

Therefore

$$T_s \approx \frac{4}{\omega_c}$$

C.3 Phase Margin

The gain crossover frequency is defined as the frequency at which

$$|G(j\omega_c)H(j\omega_c)| = 1$$

The phase margin is given by

$$PM = 180^\circ + \angle G(j\omega_c)H(j\omega_c)$$

Phase margin provides an indication of the relative stability of the closed-loop system.

C.4 Gain Margin

The phase crossover frequency is defined as the frequency at which

$$\angle G(j\omega_{pc})H(j\omega_{pc}) = -180^\circ$$

The gain margin is

$$GM = \frac{1}{|G(j\omega_{pc})H(j\omega_{pc})|}$$

or in decibels,

$$GM_{dB} = 20 \log_{10}(GM)$$

Gain margin indicates the additional gain that can be introduced before instability occurs.

Appendix D

MATLAB Scripts for Controller Design and Verification

```
clc;
clear;
close all;

%% Define Laplace Variable
s = tf('s');

%% Motor Plant (Including PWM Gain)
G = (23.21 * 0.24) / ...
    ((s/48.6482 + 1)*(s/1108.0327 + 1));

%% Controller
C = (1795.203217*(s/10 + 1)) / ...
    (s/0.002056 + 1);

%% Open-Loop Transfer Function
L = C*G;

%% Bode Plot with Gain and Phase Margins
figure;
margin(L);
grid on;
title('Open-Loop Bode Plot with Gain and Phase Margin');

%% Gain Margin, Phase Margin and Crossover Frequencies
[GM, PM, Wcg, Wcp] = margin(L);

disp(' ');
disp('===== OPEN LOOP RESULTS =====');

if isinf(GM)
    disp('Gain Margin = Infinity');
else
    disp(['Gain Margin (dB) = ', num2str(20*log10(GM))]);
end

disp(['Phase Margin (deg) = ', num2str(PM)]);

disp(['Phase Crossover Frequency (rad/s) = ', num2str(Wcg)]);
disp(['Gain Crossover Frequency (rad/s) = ', num2str(Wcp)]);

%% Closed-Loop Transfer Function
T = feedback(L,1);

%% Closed-Loop Step Response
figure;
step(T);
grid on;
title('Closed-Loop Step Response');

%% Closed-Loop Performance Parameters
info = stepinfo(T);

disp(' ');
disp('===== CLOSED LOOP STEP RESPONSE =====');

disp(['Rise Time (s) = ', num2str(info.RiseTime)]);
disp(['Settling Time (s) = ', num2str(info.SettlingTime)]);
disp(['Overshoot (%) = ', num2str(info.Overshoot)]);
disp(['Peak Time (s) = ', num2str(info.PeakTime)]);
```

```
disp(['Peak Value = ', num2str(info.Peak)]);

%% Display Transfer Functions
disp(' ');
disp('==== PLANT TRANSFER FUNCTION ====');
G

disp(' ');
disp('==== CONTROLLER TRANSFER FUNCTION ====');
C

disp(' ');
disp('==== CLOSED LOOP TRANSFER FUNCTION ====');
T
```


Appendix E

Dead-Time Generation MATLAB Function

```
function [PWM_new, NOT_PWM_new] = deadtime_logic(PWM, PWM_delay, NOT_PWM_delay)

if PWM == 1

    PWM_new = PWM_delay;
    NOT_PWM_new = ~PWM;

else

    PWM_new = PWM;
    NOT_PWM_new = NOT_PWM_delay;

end

end
```

Appendix F

Simulation Configuration

F.1 High-Accuracy Solver Configuration

Type: Variable Step

Solver: ode23t (mod. Stiff/Trapezoidal)

Max step size: 1e-6

Relative tolerance: 1e-5

The solver settings shown in Figure F.1 were used for all simulations from which quantitative performance data were extracted. A small maximum step size and tighter tolerances were employed to accurately capture switching behaviour and current dynamics.

F.2 Validation Test Solver Configuration

Type: Variable Step

Solver: ode23t (mod. Stiff/Trapezoidal)

Max step size: auto

Relative tolerance: 1e-3

For long-duration validation studies and disturbance tests, a less restrictive solver configuration was used to reduce simulation time while preserving the overall dynamic behaviour of the system.

F.3 Workspace Initialization Script

```
%% Performance Parameters from Simulink Workspace Data

% t = time vector
% w = speed vector

final_value = w(end);

% Overshoot
peak_speed = max(w);
overshoot = ((peak_speed - final_value)/final_value)*100;

% Settling Time (2% Band)
lower_limit = 0.98*final_value;
upper_limit = 1.02*final_value;

idx = find(w < lower_limit | w > upper_limit);

if isempty(idx)
```

```

        settling_time = 0;
    else
        settling_time = t(idx(end)+1);
    end

% Steady State Error
reference_speed = 300;      % Change as required

steady_state_error = ...
    abs(reference_speed - final_value);

steady_state_error_percent = ...
    (steady_state_error/reference_speed)*100;

%% Display Results

disp('-----');
disp(['Peak Speed = ', num2str(peak_speed), ' rad/s']);
disp(['Overshoot = ', num2str(overshoot), ' %']);
disp(['Steady-State Speed = ', num2str(final_value), ' rad/s']);
disp(['Steady-State Error = ', ...
    num2str(steady_state_error_percent), ' %']);
disp(['Settling Time = ', ...
    num2str(settling_time), ' s']);
disp('-----');

```